ТЕХНИЧЕСКИЙ ГАЙД

LitGraphAgent

Автономный агент для сбора, фильтрации и построения графов литературы в Obsidian

Полное введение в проект для новых участников, преподавателей и инженеров.
Архитектура, пять агентов, логика пайплайна, метрики, запуск и глоссарий.

Команда AI-RIldium | ноябрь 2026
github.com/Icold21/LitGraphAgent | MIT License

Документ объясняет не только «что сделано», но и «почему это работает»: от precision и recall до того, как LLM выбирает поисковый движок.

Содержание

1. Зачем этот документ

1.1. Что такое LitGraphAgent
1.2. Какую проблему решает проект
1.3. Для кого проект
1.4. Что система делает и чего не делает
1.5. Как читать документ
1.6. Краткая карта репозитория
1.7. Главная мысль

2. Базовые понятия без предположений

2.1. Языковая модель и её ограничения
2.2. Граф знаний
2.3. Obsidian и Markdown
2.4. Агент и оркестрация
2.5. Поиск, отбор и ранжирование
2.6. Качество, тесты и CI
2.7. Что важно понять прямо сейчас

3. Архитектура проекта

3.1. Общая картина
3.2. Почему именно пять ролей
3.3. LangGraph как диспетчер
3.4. ResearchState: что передаётся между узлами
3.5. MemorySaver и _state.json
3.6. Инварианты архитектуры
3.7. Что происходит при ошибке

4. Подробный разбор агентов

4.1. LeadAIAgent: жизненный цикл базы
4.2. RetrievalAgent: поиск, дедупликация и цитирования
4.3. PromptAgent и LLM-слой

5. Сквозные сценарии и данные

5.1. Как тема превращается в vault
5.2. Как работает обновление базы
5.3. Deep-режим и grounding
5.4. Проверка, список и удаление
5.5. Модель данных
5.6. _state.json как источник правды
5.7. Структура vault
5.8. Почему vault, а не база данных
5.9. Что происходит при повторном запуске

6. Как устроены поиск, LLM и граф

6.1. Поиск и выбор движков
6.2. Наукометрический скор
6.3. Метод локтя
6.4. LLM-слой и промпты
6.5. Граф и каноникализация

7. QA, тесты и бенчмарк

7.1. Зачем нужен отдельный QA-агент
7.2. Проверки vault
7.3. Проверки состояния
7.4. Тесты и CI
7.5. SciERC: что это и почему он релевантен
7.6. Что именно измеряет harness
7.7. Exact, relaxed и content-only
7.8. Результаты на 20 абстрактах
7.9. Что дал prompt v2

7.10. Опорные точки
7.11. Ограничения бенчмарка
7.12. Как читать эту таблицу

8. Метрики с нуля

8.1. Задача извлечения концептов
8.2. Precision
8.3. Recall
8.4. F1
8.5. Почему не accuracy
8.6. Exact и relaxed
8.7. Micro и macro
8.8. Content-only
8.9. MRR
8.10. Hits@k
8.11. nDCG
8.12. MAP
8.13. Triple F1
8.14. Coverage, orphan rate и unresolved link rate
8.15. Статистическая честность
8.16. Как читать таблицу LitGraphAgent
8.17. Частые ошибки
8.18. Главное

9. Запуск, код и команда

9.1. Установка
9.2. Настройка .env
9.3. Проверка окружения
9.4. Основные команды
9.5. Тесты и линтер
9.6. Бенчмарк
9.7. Типичные проблемы
9.8. Карта кода
9.9. Как добавить нового агента
9.10. Как добавить поисковый движок
9.11. Как изменить промпт
9.12. Команда и роли
9.13. Дорожная карта
9.14. Риски
9.15. Что считать успехом

10. Глоссарий, FAQ и приложение

10.1. Глоссарий
10.2. FAQ
10.3. Приложение A. Пример _state.json
10.4. Приложение B. Пример заметки источника
10.5. Приложение C. Пример концепт-хаба
10.6. Приложение D. Пример QA-отчёта
10.7. Приложение E. Промпт v2
10.8. Приложение F. Шпаргалка команд
10.9. Что читать дальше
10.10. Заключение

1. Зачем этот документ

Этот документ написан для человека, который впервые видит LitGraphAgent и хочет понять не только «что здесь лежит», но и «почему это работает именно так». Он рассчитан на читателя, который не обязан знать машинное обучение глубоко: все ключевые идеи — языковые модели, графы знаний, precision, recall, F1, ранжирование, метод локтя — объясняются с нуля и на примерах. При этом документ не упрощает проект до уровня презентации: он показывает реальную логику кода, формулы, ограничения и компромиссы.

Такой документ нужен потому, что проект быстро растёт. Новому участнику приходится одновременно разбираться в пяти агентах, LangGraph, Obsidian, наукометрии, промптах, JSON-устойчивости, QA и бенчмарке. Если читать только README, остаётся много «магии»: почему выбран этот движок, откуда взялся вес концепта, что именно делает метод локтя, почему exact F1 ниже relaxed и чем наш бенчмарк отличается от SciERC-лидерборда. Гайд закрывает эти разрывы.

Документ можно читать последовательно, но он устроен и как справочник. Менеджеру или преподавателю достаточно разделов 1–3, 7 и 8. Инженеру, которому нужно быстро включиться в код, полезнее разделы 3–6 и 9. Читателю, который плохо знает ML, стоит начать с разделов 2 и 8, а затем вернуться к архитектуре. Глоссарий и FAQ в конце помогают, если какой-то термин выпал из головы.

1.1. Что такое LitGraphAgent

LitGraphAgent — это автономный исследовательский агент, который получает тему и требования к литературе, сам ищет научные статьи в нескольких источниках, отбирает релевантные работы, извлекает из них ключевые концепты, строит граф связей «статья → концепт» и «статья → статья», а затем записывает результат в локальную базу Obsidian. В базе появляются атомарные заметки источников, концепт-хабы, библиография, обзорный синтез и журнал работы агентов.

Слово «автономный» здесь означает не «полностью самостоятельный разум», а «конвейер, который проходит путь от директивы до готового vault без ручного перекладывания файлов». Человек задаёт тему, требования, формат цитирования и квоту. Дальше система сама выбирает поисковые движки, собирает кандидатов, фильтрует их, ранжирует, читает абстракты (а в deep-режиме — PDF), извлекает концепты, канонизирует их, строит связи и проверяет целостность результата. Человек всё равно остаётся экспертом: он проверяет источники, читает ключевые работы и решает, можно ли доверять обзору.

Система не является поисковиком общего назначения. Она не пытается найти «всё, что есть в интернете», и не заменяет научную базу вроде Scopus. Её задача — собрать локальный, прозрачный и воспроизводимый граф литературы по конкретной теме, который можно открыть в Obsidian, дополнить руками и использовать в дальнейшей работе.

1.2. Какую проблему решает проект

У исследователя есть три повторяющиеся проблемы, и LitGraphAgent пытается закрыть их одним проходом.

Первая проблема — информационный шум. По многим темам выходят сотни и тысячи работ. Поиск по ключевым словам возвращает много нерелевантного, а ручной просмотр abstracts занимает дни. Нужен фильтр, который отсеивает мусор до того, как человек начнёт читать.

Вторая проблема — потеря связей. Даже если статьи найдены, их нужно связать между собой: какая работа использует какой метод, какой концепт встречается в нескольких статьях, кто на кого ссылается. В обычной папке с PDF эти связи теряются, и через месяц трудно вспомнить, почему статья вообще попала в подборку.

Третья проблема — рутина оформления. Библиография, единый формат цитирования, ссылки, конспекты и оглавление отнимают часы, хотя почти не требуют творчества. Если делать это вручную, легко ошибиться в годе, авторе или порядке ссылок.

LitGraphAgent строит работу так, чтобы все три проблемы решались в одном пайплайне: тема превращается в набор проверенных источников, источники — в граф концептов, а граф — в готовую библиографию и обзор. При этом каждый шаг оставляет след: какой движок выбран, какие статьи отброшены, какие концепты извлечены, какие связи построены, что нашёл QA.

1.3. Для кого проект

Проект полезен студентам и аспирантам, которые собирают литературу по новой теме и хотят получить не просто список ссылок, а структурированную карту идей. Он полезен исследователям, которые ведут локальную базу знаний и не хотят зависеть от облачного сервиса. Он полезен R&D-командам, которым нужно сохранять общий граф источников и концептов по проекту. И он полезен преподавателям, которые хотят показать студентам, как устроен полный пайплайн «поиск → отбор → извлечение знаний → граф → проверка».

Проект не требует обязательного облака. Модель может быть локальной через Ollama, а может быть внешней через DeepSeek, OpenAI, Groq, OpenRouter или любой OpenAI-совместимый endpoint. Это делает его пригодным и для машины студента, и для сервера с GPU.

1.4. Что система делает и чего не делает

Система делает пять вещей хорошо. Она ищет статьи в Semantic Scholar, arXiv и открытом вебе, причём выбор движков делает модель, а не жёстко зашитый алгоритм. Она фильтрует кандидатов по теме и требованиям, а затем ранжирует их по наукометрическим сигналам и отсекает низкоимпактный хвост. Она извлекает из абстракта summary, findings, methodology, limitations и концепты, а в deep-режиме дополнительно привязывает концепты к страницам PDF. Она канонизирует концепты, чтобы `GNN`, `GNNs` и `Graph Neural Networks` не превращались в три разных узла. И она записывает результат в Obsidian-vault: заметки, хабы, `[[wikilinks]]`, библиографию, обзор и журнал работы агентов.

Система также умеет сохранять состояние в `_state.json` и продолжать работу через `--update`, не перезаписывая существующие определения. Она проверяет целостность vault и состояния через QA-агент и измеряет качество извлечения концептов на SciERC.

При этом система не гарантирует, что источник качественный или peer-reviewed. Она не заменяет чтение статьи экспертом. Она не извлекает типизированные отношения уровня SciERC, потому что пайплайн их не предсказывает; это ограничение честно зафиксировано в бенчмарке. Она не выполняет end-to-end оценку качества поиска вроде precision@k или nDCG — это задача следующего этапа. И она не является облачным сервисом: все данные лежат локально, а качество зависит от выбранной модели.

1.5. Как читать документ

Первые два раздела дают контекст и словарь. Если вы уже знаете, что такое LLM, промпт, граф знаний и Obsidian, раздел 2 можно просмотреть по диагонали, но к нему стоит возвращаться, когда термины появляются в коде.

Раздел 3 описывает общую архитектуру: пять ролей, LangGraph, `ResearchState`, `MemorySaver` и `_state.json`. Здесь важно понять главную мысль: LangGraph — это диспетчер, а не «мозг». Бизнес-логика живёт в агентах, а граф только маршрутизирует шаги и ведёт журнал передач.

Раздел 4 — самый подробный разбор агентов. Для каждого агента объясняется, что он получает, что делает по шагам, что отдаёт, где лежит код и какие у него ограничения. Если вам нужно быстро понять, кто за что отвечает, начните с таблицы ролей в разделе 3, а затем читайте раздел 4.

Раздел 5 собирает сквозные сценарии: create, update, deep, check, list, delete. Здесь же описаны модель данных, `_state.json`, структура vault и формат заметок. Это полезно, если вы хотите понять, как система хранит информацию между запусками.

Раздел 6 объясняет три «начинки»: поиск и ранжирование, LLM-слой и граф. Именно здесь находятся формулы наукометрического скора, метод локтя, алгоритм канониkализации и правила записи заметок.

Раздел 7 посвящён QA, тестам, CI и бенчмарку SciERC. Здесь есть результаты, опорные точки, объяснение prompt v2 и честные ограничения.

Раздел 8 — самостоятельный курс по метрикам. Он объясняет precision, recall, F1, micro/macro, exact/relaxed, MRR, Hits@k, nDCG, MAP и triple F1 на примерах. Если вы плохо знаете ML, это лучший раздел для входа.

Раздел 9 — практика: установка, `.env`, CLI, типичные проблемы, карта кода, роли и дорожная карта. Раздел 10 — глоссарий, FAQ и приложение с примерами файлов.

1.6. Краткая карта репозитория

Код организован по зонам ответственности, а не по типам файлов. В папке `lit_graph/agent/` лежат роли, оркестрация, состояние и агенты. В `lit_graph/search/` — клиенты arXiv, Semantic Scholar и веба, каталог движков, роутер, ранжирование и извлечение PDF. В `lit_graph/llm/` — провайдеры и все промпты. В `lit_graph/storage/` — запись Obsidian-vault, имена файлов и состояние. В `lit_graph/qa/` — проверки vault и состояния. В корне пакета лежат `cli.py`, `config.py` и `models.py`.

Отдельно стоят `tests/`, `benchmarks/` и `docs/`. В `tests/` — 55 тестов, которые покрывают pipeline, агентов, персистентность, канониkализацию, QA, батчинг и провайдеры. В `benchmarks/` — SciERC-бенчмарк, сырые результаты и скрипт построения графиков. В `docs/` — архитектура, отчёты, презентации, метрики и этот гайд.

```
lit_graph/
  agent/        роли, оркестрация, состояние, агенты
  search/       клиенты arXiv/S2/web, каталог, роутер, ранжирование
  llm/          провайдеры LLM и все промпты
  storage/      запись Obsidian vault, состояние, имена файлов
  qa/           проверки vault и состояния
  cli.py        командная строка
  config.py     настройки из .env
  models.py     Paper, ConceptHub, GraphEdge, LiteratureBaseState
tests/          55 тестов
benchmarks/     SciERC-бенчмарк и результаты
docs/           архитектура, отчёты, презентации, гайд
```

1.7. Главная мысль

LitGraphAgent — это не «магический ИИ, который всё знает». Это конвейер, в котором каждый шаг можно проверить: какой движок выбран, какие статьи отброшены, какие концепты извлечены, какие связи построены, что нашёл QA. Именно прозрачность шагов делает систему пригодной для учебного и исследовательского контекста. Если вы запомните только одну мысль из этого гайда, пусть это будет она: качество здесь рождается не из одной большой модели, а из дисциплины пайплайна — валидации, каноникализации, батчинга, проверок и честных метрик.

2. Базовые понятия без предположений

Этот раздел объясняет слова, которые дальше встречаются постоянно. Его можно читать как небольшой учебник: сначала о языковых моделях, затем о графах знаний, Obsidian, агентах и качестве. Если какой-то термин кажется очевидным, его можно пропустить, но лучше вернуться к нему, когда он появится в коде.

2.1. Языковая модель и её ограничения

Большая языковая модель, или LLM, — это функция, которая получает текст и предсказывает продолжение. Внутри неё миллионы или миллиарды параметров, обученных на больших корпусах. Модель не хранит факты как таблицу в базе данных. Она выучила статистические закономерности языка и мира, поэтому может уверенно отвечать правильно, а может так же уверенно ошибаться. Это важно понять сразу: LLM — вероятностный инструмент, а не оракул.

В LitGraphAgent модель используется не для свободного чата, а для конкретных операций. Она читает описание поисковых движков и выбирает подходящие. Она проверяет, соответствует ли статья теме и требованиям. Она извлекает из абстракта summary, findings, methodology, limitations и концепты. Она пишет определения концептов, собирает библиографию и обзор. В каждом случае у модели есть роль, входные данные, правила и ожидаемый формат ответа.

Промпт — это текст, который мы отправляем модели. Промпт не равен вопросу в бытовом смысле. Это инструкция: кто ты, что тебе дано, что нужно сделать, в каком формате ответить и чего нельзя делать. В проекте почти все промпты требуют строгий JSON, потому что результат должен разбираться программно. Если модель добавит markdown-обёртку, комментарий или незакрытую скобку, пайплайн должен это пережить.

Токен — минимальная единица текста для модели. Это может быть слово, часть слова или знак. Модель генерирует токены по одному, поэтому у неё есть ограничение на длину контекста и на длину ответа. Если положить в контекст слишком много текста, модель может потерять важное или упереться в лимит. Если попросить слишком длинный ответ, он может обрезаться. Именно поэтому в проекте есть батчинг: концепты определяются порциями, библиография — порциями, а не одним гигантским запросом.

Контекст — это весь текст, который модель видит в одном запросе: системная инструкция, промпт, фрагмент статьи. Чем больше контекст, тем больше у модели шансов запутаться, поэтому в промптах есть ограничения на длину абстракта и на количество концептов за раз. Температура — параметр случайности. Чем она выше, тем разнообразнее ответы; чем ниже, тем стабильнее. В проекте используется низкая температура 0.2, чтобы ответы были предсказуемее, но полной детерминированности всё равно нет.

JSON — текстовый формат вида `{"key": "value"}`. Он удобен тем, что его можно строго разобрать в программе. Но модель может вернуть не идеальный JSON: добавить `<think>...</think>`, обернуть в ```json, поставить комментарий или обрезать скобку. Поэтому в проекте есть отдельный санитайзер, который пытается восстановить структуру. Он не делает модель умнее, но спасает от типичных локальных моделей.

Ollama — локальный сервер, который запускает модели на вашем компьютере и предоставляет OpenAI-совместимый API. В проекте через Ollama запускаются `qwen2.5:3b` и `qwen3.5:4b`. Локальная модель не отправляет текст в облако, но требует GPU/CPU и времени. На слабом ноутбуке 4B-модель может

выдавать один- два токена в секунду, и один абстракт будет обрабатываться минуты. Это не ошибка проекта, а свойство локального инференса.

Провайдер LLM — конкретный сервис или локальный сервер, к которому обращается проект. Поддерживаются DeepSeek, OpenAI, Groq, OpenRouter, Ollama, vLLM и любой OpenAI-совместимый endpoint. Провайдер скрыт за общим интерфейсом, поэтому замена модели — это изменение `.env`, а не переписывание агентов.

Эмбеддинги — числовые векторы, которые представляют смысл текста. В этом проекте они не используются для поиска. Поиск идёт через API научных баз, а связи строятся по концептам и цитированиям. Это важно, чтобы не искать «векторную магию» там, где её нет.

2.2. Граф знаний

Граф — это структура из узлов и рёбер. Узел обозначает объект, ребро — связь. Граф знаний — это граф, где узлы обозначают сущности, а рёбра — отношения между ними. Часто граф знаний представляют в виде троек:

```
(голова, отношение, хвост)
(Induction Head, part_of, Transformer Circuits)
(Paper A, cites, Paper B)
```

Узел в LitGraphAgent — это источник или концепт. Источники лежат в `Sources/`, концепты — в `Concepts/`. Ребро `paper → concept` означает, что статья упоминает концепт. Ребро `paper → paper` означает цитирование и имеет тип `cites`. Обратное ребро `concept → paper` генерируется в концепт-хабе, чтобы из концепта можно было перейти к источникам.

Триплет — это тройка «голова, отношение, хвост». В SciERC и других KG-бенчмарках качество часто измеряют именно на тройках. Но в нашем бенчмарке типизированные отношения не оцениваются, потому что пайплайн их не предсказывает. Мы оцениваем узлы-концепты и бипартитные рёбра `paper → concept`. Это честнее, чем считать relation F1 на данных, которых система не производит.

Онтология — это правила, какие типы узлов и рёбер вообще допустимы. В проекте онтология задана структурой vault и типами заметок: `literature_note`, `concept_hub`, `external_reference`, `synthesis_hub`, `master_bibliography`, `agent_activity`. Если заметка имеет неизвестный `type`, QA выдаст warning.

Каноникализация — это приведение разных написаний одной сущности к одному каноническому виду. Например, `GNN`, `GNNs`, `Graph Neural Networks` должны стать одним концептом. Без каноникализации граф распадается на дубли, и пользователь видит три узла вместо одного. В проекте каноникализация эвристическая: она снимает пунктуацию, раскрывает известные акронимы, singularizes последнее слово и выбирает самый частый display-вариант. Этого достаточно для большинства случаев, но не для всех.

Покрытие, или coverage, — доля gold-сущностей или связей, которые попали в граф. Если в статье десять концептов, а система извлекла пять, coverage равен 0.5. Покрытие полезно, но не заменяет precision и recall: можно покрыть всё, но с кучей мусора.

Сирота, или orphan, — узел без входящих и исходящих ссылок. В Obsidian это заметка, до которой нельзя дойти по ссылкам. Unresolved link — это `[[ссылка]]`, которая не находит заметку. В Obsidian такая ссылка выглядит как «битая». QA проверяет и то, и другое.

2.3. Obsidian и Markdown

Markdown — простой текстовый формат разметки. Заголовки обозначаются #, списки — -, выделение — **жирным**, ссылки — [текст](url). Markdown удобен тем, что остаётся читаемым даже без рендера.

Obsidian — приложение для ведения заметок в виде локальной папки Markdown-файлов. Оно показывает граф связей и поддерживает [[wikilinks]]. Vault — это папка с заметками Obsidian. В LitGraphAgent vault является результатом работы: Sources/, Concepts/, библиография, обзор, состояние.

Frontmatter — YAML-блок в начале заметки между ---. В нём хранятся метаданные: title, type, year, citation_count, node_weight и другие. Frontmatter нужен, чтобы Obsidian и QA могли понимать тип заметки и её свойства без разбора всего текста.

Wikilink — ссылка вида [[Название заметки]]. Она связывает заметки внутри vault и отображается в graph view. Graph view — режим Obsidian, где заметки показаны узлами, а ссылки — рёбрами. Именно его пользователь видит как «карту литературы». Stub, или заглушка, — лёгкая заметка для внешней статьи, которая упоминается в цитированиях, но не была загружена. Она нужна, чтобы [[wikilink]] не повисал.

2.4. Агент и оркестрация

Агент — это программный компонент, который выполняет ограниченную роль: получает данные, принимает решение, возвращает результат. В LitGraphAgent пять ролей: Lead, Retrieval, Prompt, Graph и QA. Это не пять независимых ботов, которые переписываются между собой. Это пять наборов методов, которые вызываются из одного workflow. Роли разделены по ответственности, а не по процессам.

Оркестрация — это управление тем, в каком порядке агенты вызываются и как передаются данные. В проекте оркестратор — LangGraph. LangGraph — библиотека для описания workflow как графа: узлы — шаги, рёбра — переходы, условные рёбра — ветвления. Важно: LangGraph не содержит бизнес-логику. Он только маршрутизирует вызовы агентов. Если узел вызывает retrieval.search, то именно RetrievalAgent решает, как искать, а не LangGraph.

Состояние, или state, — данные, которые передаются между узлами. В проекте это ResearchState: тема, требования, кандидаты, выбранные статьи, текущая статья, страницы PDF, activity и QA-отчёт. Reducer — функция, которая объединяет старое и новое значение поля. Для activity используется append-reducer: каждый узел добавляет событие, а не перезаписывает журнал. Checkpoint — сохранённая точка состояния. MemorySaver хранит состояние конкретного запуска в памяти; _state.json — durable-хранилище между запусками.

2.5. Поиск, отбор и ранжирование

Retrieval, или поиск, — получение кандидатов из внешних источников. Semantic Scholar — академическая поисковая база с метаданными и цитированиями; она даёт citation count и influential citation count. arXiv — preprint-сервер, хороший для свежих работ, но не гарантирующий peer-review. Open web — веб- поиск и парсинг страниц; он нужен, когда академические API не дают результат, но качество там шумнее.

Дедупликация — удаление одинаковых статей, пришедших из разных движков. Сравниваются DOI, URL и нормализованное название. Ранжирование — сортировка кандидатов по полезности. В проекте используется наукометрический скор, который учитывает citation velocity, influential ratio и log-scaled impact. Подробная формула и пример — в разделе 6.

Elbow cutoff, или метод локтя, — эвристика, которая делит отсортированные статьи на сильную голову и слабый хвост. Хвост отсекается. Это не «магия», а способ не тащить в базу случайные low-impact работы. Подробный разбор метода с формулами и примером — в разделе 6.

Fallback — запасной вариант. Если primary-движок дал мало результатов, система пробует fallback-движок. Sub-query — уточняющий подзапрос, который генерирует LLM, если после первого раунда статей недостаточно.

2.6. Качество, тесты и CI

QA, или quality assurance, — проверка результата: ссылки, frontmatter, дубли, сироты, целостность связей. QA-агент не мутирует данные. Он читает vault и состояние, собирает issues с severity и отдаёт отчёт.

Тест — автоматическая проверка поведения кода. В проекте 55 тестов. pytest — фреймворк для тестов на Python. ruff — быстрый линтер и форматтер; в CI он падает, если есть ошибки стиля. CI, или continuous integration, — автоматический запуск тестов и линтера на каждый push/pull request. В проекте GitHub Actions проверяет Python 3.10–3.13.

Бенчмарк — стандартизированный набор данных и протокол оценки. В проекте используется SciERC. Precision, recall и F1 — метрики качества. Интуиция такая: precision — «сколько из найденного правильное», recall — «сколько из правильного найдено», F1 — их баланс. Подробный разбор с формулами и примерами — в разделе 8.

2.7. Что важно понять прямо сейчас

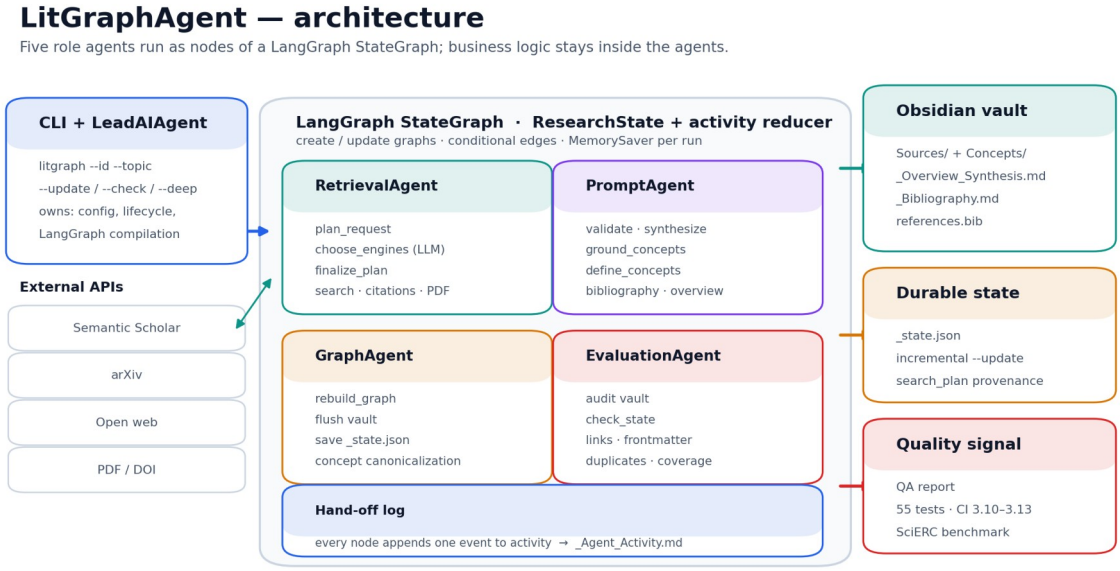
Первое: LLM — вероятностный компонент, а не база фактов. Поэтому в проекте есть валидация, JSON-санитайзер, fallback-и и QA. Второе: граф знаний — это узлы и рёбра, а не просто папка с файлами. Поэтому важны каноникализация, ссылки и дедупликация. Третье: LangGraph — это диспетчер, а не «мозг». Бизнес-логика живёт в агентах. Четвёртое: `_state.json` — источник правды между запусками; без него `--update` не смог бы продолжать базу. Пятое: бенчмарк измеряет только то, что система реально предсказывает. Если пайплайн не предсказывает типы отношений, relation F1 не считается.

Если вы поняли эти пять мыслей, дальше будет намного проще: архитектура, агенты и код начнут складываться в единый механизм, а не в набор разрозненных файлов.

3. Архитектура проекта

3.1. Общая картина

LitGraphAgent построен как команда из пяти ролей, которые работают в одном пайплайне. Пользователь запускает CLI, Lead-агент собирает LangGraph-workflow и передаёт работу специалистам. Каждый шаг — отдельный узел графа; каждый переход — ребро; каждый вызов агента оставляет событие в журнале `activity`. В конце vault записывается на диск, а QA-агент проверяет, что ссылки, заметки и состояние не противоречат друг другу.



Архитектура LitGraphAgent

Пять ролей — это не декоративная схема, а способ разделить ответственность. Lead отвечает за жизненный цикл базы и оркестрацию. Retrieval отвечает за поиск, дедупликацию, цитирования и PDF. Prompt отвечает за всё, что связано с LLM: валидацию, синтез, концепты, библиографию и обзор. Graph отвечает за онтологию, каноникализацию и запись Obsidian. QA отвечает за независимую проверку результата. Если менять один слой, остальные остаются на месте: например, можно заменить arXiv-клиент, не трогая промпты, или добавить проверку QA, не меняя граф.

Роль	Агент	Что делает
Lead AI Engineer	`LeadAIAgent` (`LiteratureAgent`)	Жизненный цикл базы, сборка workflow, CLI, состояние
Data / IR Engineer	`RetrievalAgent`	Поиск, дедупликация, цитирования, PDF, кэш
LLM Engineer	`PromptAgent`	Валидация, синтез, концепты, библиография, обзор
Graph & Knowledge Engineer	`GraphAgent`	Онтология, каноникализация, запись vault, ссылки
QA / Evaluation Engineer	`EvaluationAgent`	Аудит vault и состояния, отчёт QA

3.2. Почему именно пять ролей

Разделение возникло из реальных зон. Поиск требует знания API, ретраев, дедупликации и ранжирования; это отдельная инженерная зона, которая быстро меняется при добавлении нового источника. LLM-операции требуют промптов, батчинга, JSON-устойчивости и валидации; это другая зона, где ошибки связаны с качеством модели, а не с сетью. Граф требует онтологии, каноникализации, имён файлов и Obsidian; это третья зона, где важны детерминизм и совместимость с пользовательскими настройками. QA должен быть независимым: он не пишет данные, а проверяет то, что написали другие. Lead координирует, но не подменяет специалистов.

Такое разделение даёт три практических выигрыша. Во-первых, тесты можно писать по слоям: retrieval тестируется отдельно от промптов, а граф — отдельно от QA. Во-вторых, ошибку легче локализовать: если статья не прошла отбор, это Prompt/Validator, а если ссылка не находится в Obsidian — Graph/QA. В-третьих, новый участник может начать с одной зоны и не держать в голове весь проект сразу.

3.3. LangGraph как диспетчер

`LiteratureWorkflow` (`lit_graph/agent/workflow.py`) собирает `StateGraph(ResearchState)`. Каждый узел — это обёртка `_step(recipient, action, fn)`, которая вызывает метод агента, замеряет время, добавляет событие в `activity` и возвращает только изменённые ключи состояния.

Такая обёртка решает три задачи. Во-первых, она даёт единый журнал: не нужно помнить, какой агент что сделал, — каждое событие имеет отправителя, получателя, действие, статус, timestamp и длительность. Во-вторых, она дисциплинирует узлы: узел не может вернуть произвольный словарь, он возвращает только те ключи `ResearchState`, которые действительно изменил. В-третьих, она отделяет бизнес-логику от маршрутизации: узел вызывает `self.retrieval.search(...)`, а не реализует поиск внутри себя.

Схема режима `create` выглядит так:

```
START
  │
  ├─ plan_request            (решаем, нужен ли LLM для выбора движков)
  ├─ choose_engines          (LLM выбирает primary/fallback)
  ├─ finalize_plan           (SearchRouter валидирует и сохраняет план)
  ├─ search                  (primary + fallback, дедупликация)
  ├─ validate                (LLM проверяет тему и требования)
  │
  ├─ если мало статей: suggest_queries → search → validate (до 3 раундов)
  │
  ├─ rank_select             (наукометрия + elbow cutoff)
  ├─ expand_citations        (references/citations для текущей статьи)
  ├─ synthesize              (summary, findings, methodology, limitations, concepts)
  │
  ├─ если --deep: fetch_pages → ground_concepts
  │
  ├─ commit_paper            (статья попадает в base)
  │
  ├─ следующая статья: снова expand_citations → ... → commit_paper
  │
  ├─ rebuild_graph           (концепты, веса, рёбра)
  ├─ define_concepts         (определения концептов батчами)
  ├─ bibliography            (форматированная библиография)
  ├─ link_network            (типизированные связи)
  ├─ overview                (обзорный синтез)
  ├─ flush                   (запись vault)
  └─ audit                   (QA)
END
```

Смысл этой схемы важнее, чем её внешний вид. Сначала система планирует поиск, затем ищет и валидирует, затем при необходимости повторяет поиск с подзапросами. После того как набрано достаточно статей, она ранжирует их и выбирает голову списка. Дальше начинается ingestion: для каждой статьи подтягиваются цитирования, делается синтез, в deep-режиме — PDF и grounding, и только потом статья коммитится в состояние. Когда все статьи обработаны, строится граф, определяются концепты, собирается библиография, строятся связи, пишется обзор, vault сохраняется и проверяется QA.

Режим `update` короче. После `validate` уже известны новые статьи, поэтому сразу идёт `expand_citations`, затем синтез и commit, а в конце — `save_state` и `audit`. Существующие определения концептов не перезаписываются. Это позволяет расширять базу неделями, не теряя предыдущую работу.

3.4. ResearchState: что передаётся между узлами

`ResearchState` (`lit_graph/agent/graph_state.py`) — это `TypedDict`, в котором лежат только JSON-сериализуемые значения. В нём есть идентификатор запуска и режим, тема и требования, квота и deep-флаг, вся литературная база как dict, поисковые раунды, кандидаты, выбранные статьи, текущая статья и страницы PDF, результаты графа, QA-отчёт и журнал активности.

Ключевая деталь: runtime-объекты (LLM-клиент, vault, агенты) не лежат в состоянии. Они живут на объекте workflow. Это важно, потому что состояние должно быть сериализуемым: иначе checkpoint LangGraph и `_state.json` сломаются. Если бы в state лежал открытый клиент LLM или файловый дескриптор, его нельзя было бы сохранить и восстановить.

Поле `activity` использует `Annotated[..., merge_activity]`. Это reducer, который склеивает списки событий от разных узлов. Каждый узел добавляет одно событие, а LangGraph накапливает их. Именно этот журнал потом превращается в `_Agent_Activity.md`: пользователь видит, какой агент что сделал, в каком порядке и сколько времени это заняло.

3.5. MemorySaver и `_state.json`

В проекте два уровня состояния, и их легко перепутать. `MemorySaver` — это checkpoint LangGraph на время одного запуска. Он хранит промежуточные значения графа и позволяет переживать шаги. `_state.json` — это durable-хранилище между запусками. Оно лежит в vault и содержит `LiteratureBaseState`: статьи, концепты, рёбра, библиографию, `search_plan` и `activity`.

`MemorySaver` не заменяет `_state.json`. Если процесс упал, `_state.json` останется; если запуск завершён, именно он позволяет `--update` продолжить базу. Можно думать об этом так: `MemorySaver` — оперативная память запуска, а `_state.json` — долговременная память проекта.

Запись `_state.json` атомарная. Сначала пишется `.json.tmp`, затем файл заменяется. Это защищает от ситуации, когда прерванный запуск оставил битый JSON. Если замена не удалась, временный файл удаляется, а старый остаётся целым. Это маленькая деталь, но именно она отличает «игрушечный скрипт» от инструмента, которым можно пользоваться повторно.

3.6. Инварианты архитектуры

При доработке проекта важно соблюдать несколько правил. Специалист не пишет данные другого специалиста напрямую: только через свой агент. Все промпты живут в `llm/prompts.py`; если промпт

продублирован inline, изменения в одном месте не влияют на другое, и бенчмарк начинает врать. Все записи vault идут через `GraphAgent` и `ObsidianVaultManager`: это единственный способ гарантировать имена файлов, YAML и ссылки. QA не мутирует состояние, только читает и сообщает. Узел возвращает только те ключи `ResearchState`, которые он действительно изменил. Каждый запуск заканчивается сохранённым `_state.json`. Новый агент добавляется ролью, сервисом, узлом и тестом, а не правкой «на месте» в середине pipeline.

Эти инварианты не бюрократия. Они снижают вероятность того, что через месяц кто-то добавит новый шаг, который пишет в vault напрямую, и сломает каноникализацию или QA. Архитектура держится на договорённостях, и эти договорённости записаны в коде и документации.

3.7. Что происходит при ошибке

Ошибка одного узла логируется и пробрасывается: workflow не должен молча продолжать с битым состоянием. Ошибка одного абстракта в бенчмарке не роняет весь прогон: она попадает в `failures`, а бенчмарк идёт дальше. Ошибка сети в retrieval не роняет запуск: клиент делает retry/backoff, а кандидат может быть пропущен. Ошибка JSON в LLM-ответе пытается восстановиться санитайзером, затем retry, затем fallback. QA-ошибки не отменяют запись vault, но попадают в отчёт и CLI.

Именно эта комбинация «жёсткие инварианты + мягкая деградация» делает систему пригодной для реальных запусков, где сеть и локальная модель ведут себя неидеально. Жёсткие инварианты защищают целостность базы; мягкая деградация не даёт одному сбою уничтожить весь результат.

4. Подробный разбор агентов

В этом разделе каждый агент описан как механизм: что он получает, какие преобразования выполняет, что отдаёт, где лежит код и какие у него слабые места. Читать его лучше после раздела 3, чтобы понимать, как агенты связаны через `ResearchState`.

4.1. LeadAIAgent: жизненный цикл базы

LeadAIAgent, или `LiteratureAgent`, — это «продюсер» проекта. Он не ищет статьи и не пишет заметки вручную; он собирает workflow, запускает его, печатает прогресс, сохраняет итог и показывает QA. Его код лежит в `lit_graph/agent/core.py`, а роль описана в `agent/roles.py`.

При создании базы Lead получает `base_id`, тему, требования, формат цитирования, путь к vault, квоту статей и deep-флаг. Он открывает vault, создаёт `GraphAgent`, формирует пустой `LiteratureBaseState` с `created_at`/`updated_at` и собирает начальный `ResearchState`. В этом состоянии уже есть режим `create`, тема, требования, `queries=[topic]`, пустые кандидаты и пустой `seen_ids`. Дальше Lead собирает `LiteratureWorkflow`, компилирует граф с `MemorySaver` и запускает `graph.stream(...)`.

Во время выполнения Lead обновляет прогресс-бар по имени узла. Это не косметика: по прогрессу видно, на каком этапе находится запуск — поиск, валидация, синтез, запись vault или QA. Когда граф завершён, Lead забирает финальное состояние через `graph.get_state(config).values`, преобразует `final["base"]` обратно в `LiteratureBaseState` и печатает таблицу выбранных статей: ранг, название, цитирования, score и weight. Затем он печатает итоговую панель с путём к vault, числом источников, концептов и связей и показывает QA-отчёт.

При обновлении базы Lead делает почти то же самое, но с несколькими отличиями. Он биндит существующий vault к `GraphAgent`, обновляет `updated_at`, формирует `combined_topic = state.topic + directive` и запускает workflow в режиме `update` с `queries=[directive]`. После завершения он считает, сколько статей добавилось, и сообщает результат. Ключевая деталь: Lead не удаляет старые статьи и не перезаписывает существующие определения. Он добавляет новое, а `GraphAgent` сохраняет старое.

Методы Lead перечислены в коде, но важнее понять их смысл. `initialize_base(...)` создаёт базу, `update_base(...)` расширяет её, `flush_to_vault(...)` принудительно записывает vault, `team_table()` печатает таблицу ролей для `litgraph --team`, а `_run_graph(...)` — общая сборка и запуск workflow. Вспомогательные `_display_plan(...)` и `_report_qa()` нужны для человекочитаемого вывода.

Слабое место Lead в том, что он не валидирует научное качество сам. Это делают Prompt и QA. Он также не гарантирует, что статей ровно `max_papers`: validation и elbow могут выбрать меньше. И он не спасает от плохой модели: если LLM стабильно ошибается, это видно в QA и метриках, но автоматически не исправляется. Lead — это дирижёр, а не универсальный исправляющий слой.

4.2. RetrievalAgent: поиск, дедупликация и цитирования

RetrievalAgent отвечает за данные. Его код — `agent/retrieval_agent.py`, а клиенты — в `lit_graph/search/`. Роль — Data / IR Engineer. Задача агента: найти кандидатов, не потерять метаданные, не натащить дубликатов и подготовить данные для валидации и ранжирования.

Начинается всё с каталога движков. В `search/tools.py` каждый движок описан как данные: ключ, название, краткое описание, сильные и слабые стороны, лучшие сценарии. `catalog()` возвращает список для CLI `--tools`, а `catalog_prompt()` — текст для LLM, чтобы модель выбирала движки не «из

головы», а из описаний. Это важный архитектурный выбор: каталог — это данные, а не ветвление кода. Новый движок добавляется описанием и клиентом, а не условием «если движок X» внутри агента.

Метод `plan_request(task, requirements, context)` решает, нужен ли LLM для выбора движков. Если пользователь передал `--engines`, LLM не нужен: план строится вручную. Если `--no-router`, используется эвристика. Иначе возвращается `needs_llm=True`, и workflow идёт в `choose_engines`. Затем `finalize_plan(decision, task, requirements, source)` вызывает `SearchRouter`, который нормализует ключи, отбрасывает неизвестные, ограничивает число primary, подбирает fallback и сохраняет rationale и source. Если ответ модели невалиден, router строит эвристический план. План записывается в `base.search_plan` и печатается пользователю.

Метод `search(query, top_k, engines, fallback_engines, min_results)` выполняет основную работу. Сначала он ищет в primary-движках. Если результатов меньше `min_results`, добавляет fallback. Затем объединяет метаданные и дедуплицирует по DOI, URL и названию. Возвращает кандидатов с `source_engine`, чтобы в заметке было видно, откуда пришла статья.

В workflow поиск повторяется до трёх раундов. Если статей мало, LLM генерирует sub-queries, и цикл `search → validate` повторяется. `seen_ids` не даёт бесконечно возвращать одних и тех же кандидатов. Это важно, потому что разные запросы могут пересекаться, и без `seen_ids` система тратила бы время на уже обработанные статьи.

`merge_metadata(existing, incoming)` аккуратно сливает записи: не затирает непустые поля пустыми, дополняет отсутствующие. Это нужно, потому что arXiv может дать abstract, а Semantic Scholar — citation counts. Если просто перезаписать запись последним ответом, можно потерять полезные метаданные.

`expand_citations(paper_id, limit)` получает references и citations. В workflow они превращаются в рёбра `cites` и сохраняются в `Paper`: `referenced_paper_titles` и `citing_paper_titles`. Внешние работы, которых нет в базе, позже получат stub-заметки. Это нужно, чтобы граф цитирований не терял связи и чтобы `[[wikilinks]]` не повисали.

`fetch_pages(doi_or_url, pdf_url)` скачивает open-access PDF, извлекает текст постранично и возвращает `{page: text}`. Используется только в `--deep`. Если PDF нет или он защищён, deep-режим деградирует: grounding пропускается, статья всё равно попадает в базу.

Сеть — главный источник нестабильности. Поэтому `utils/http.py` и `utils/cache.py` дают таймауты, retry/backoff для 429/5xx, дисковый кэш поисков и цитирований, TTL и отключение через `--no-cache`. Кэш важен не только для скорости, но и для воспроизводимости: повторный запуск не дёргает API заново и не зависит от текущей нагрузки внешнего сервиса.

Ограничения RetrievalAgent очевидны. Качество зависит от внешних API; при их недоступности база будет меньше. Open web шумный; его спасает валидация и ранжирование. Citation counts могут быть неполными или устаревшими. Дедупликация по названию может склеить разные работы с одинаковым названием, поэтому DOI и URL имеют приоритет.

4.3. PromptAgent и LLM-слой

PromptAgent — это весь LLM-слой. Его код — `agent/prompt_agent.py`, `agent/validator.py`, `agent/synthesizer.py` и `lit_graph/llm/`. Роль — LLM Engineer. Задача: превратить «сырой» текст статьи в структурированные данные и определения, устойчиво к плохому JSON и ограничениям локальной модели.

Провайдеры лежат в `llm/base.py` и `llm/providers.py`. Базовый интерфейс задаёт `generate` и `generate_json`, а реализация работает с любым OpenAI-совместимым API. Поддерживаются DeepSeek, OpenAI, Groq, OpenRouter, Ollama и vLLM. Переключение — через `.env`. Провайдер отвечает за таймауты, повторы, температуру и опциональный кэш ответов по хэшу промпта.

Валидация выполняется `LiteratureValidator.validate_candidate(paper, topic, requirements)`. Это один консолидированный LLM-проход, который проверяет тему, требования и научную ценность. Результат пишется в `Paper`: `topic_relevance_score`, `topic_relevance_reason`, `requirements_met`, `requirements_reason`, `scientific_value_score`. Кандидат, не прошедший порог, не попадает в выборку, но причина остаётся в логах и состоянии для отладки. Один проход дешевле и меньше рассинхронизирует два отдельных проверяющих вызова.

Синтез выполняет `LiteratureSynthesizer.synthesize_paper_content(paper)`. Он вызывает `paper_synthesis_prompt` и заполняет `summary_en`, `key_findings_en`, `methodology_en`, `limitations_en`, `extracted_concepts` и `extracted_concepts_all`. Первый список усечён до `MAX_CONCEPTS_PER_PAPER`, второй хранит все концепты до усечения, чтобы повторная канониkализация при `--update` не теряла идеи.

Раньше `paper_synthesis_prompt` существовал, но `synthesizer.py` собирал inline-промпт. Это означало, что изменения в `llm/prompts.py` не влияли на синтез. Дубликат убрали, и теперь формулировка живёт в одном месте. Это не косметический рефакторинг: именно после него стал возможен корректный A/B промптов и прирост метрик, описанный в разделе 7.

JSON-устойчивость — отдельный слой. `_sanitize_and_parse_json` сначала убирает `<think>` и `<thought>`-блоки, затем снимает markdown-обёртку

```
валидный объект или массив в тексте. Если и это не удалось, возвращается `{}`
и пишется warning. Это не «делает модель умнее», но спасает от типичных
локальных моделей, которые добавляют комментарии или обрезают JSON.

Батчинг нужен, чтобы ответ не обрезался по лимиту токенов.
`CONCEPT_BATCH_SIZE` задаёт, сколько концептов определять за один запрос, а
`BIBLIOGRAPHY_BATCH_SIZE` — сколько источников форматировать за раз. Без
батчинга большие базы получали бы половину пустых определений.

Определения концептов делает `synthesize_all_concept_hubs_batch(state)`. Он
берёт концепты без определения и просит модель написать два предложения.
Определения сохраняются в `ConceptHub.summary_en`. При `--update`
существующие определения не перезаписываются. Это важно: пользователь мог
отредактировать определение руками, и система не должна затирать его.

Библиография, связи и обзор — отдельные методы. `harmonize_master_bibliography`
приводит цитаты к единому формату. `link_paper_network` строит типизированные
связи между статьями. `build_overall_synthesis` пишет общий обзор.
`ground_concepts_to_pages` привязывает концепты к страницам PDF в deep-режиме.
Каждый из этих методов использует тот же провайдер и тот же санитайзер, но
свои промпты и свои правила.

Ограничения LLM-слоя честно зафиксированы. Локальные модели 3–4B путают
термины и иногда добавляют лишние концепты. JSON-санитайзер восстанавливает
структуру, но не факты. Промпт не заменяет эксперта: summary и findings нужно
проверять. Определения концептов могут быть общими, если в базе мало
контекста. Именно поэтому QA и бенчмарк существуют отдельно от LLM.

## 4.4. GraphAgent: онтология и запись vault

GraphAgent — это слой знаний. Его код — `agent/graph_agent.py`,
`agent/concepts.py` и `storage/`. Роль — Graph & Knowledge Engineer. Задача:
превратить состояние в связный Obsidian-vault и не потерять пользовательские
данные.

`rebuild_graph(state, papers)` собирает концепты из статей, канонизирует имена,
```

считает веса и aggregated score, строит связи `paper → concept` и `concept → paper`, сохраняет `state.concepts` и `state.edges` и возвращает статистику. Это не просто «записать файлы»: сначала нужно понять, какие концепты вообще существуют и как они связаны.

Каноникализация — сердце GraphAgent. `concept_key(name)` приводит имя к ключу сравнения: убирает пунктуацию, раскрывает известные акронимы, снимает parenthetical-часть, если она является сокращением, и singularizes последнее слово с учётом uncountable-слов. `display_name(name)` делает человекочитаемое имя: smart title case, сохранение акронимов и mixed-case. `choose_display_names(names)` выбирает самый частый вариант, а `dedupe_concepts(names)` убирает дубли.

Пример: `Graph Neural Networks (GNNs)` превращается в ключ `graph neural network` и display `Graph Neural Network`. `GNNs` превращается в тот же ключ. `GNN` и `Graph Neural Network` сливаются. Без этого в графе было бы три узла вместо одного.

Вес концепта зависит от числа упоминаний и наукометрического веса статей. Нормализация в 0..1 позволяет использовать размер узла в graph view. `MIN_CONCEPT_PAPERS` отсекает концепты, которые встретились меньше чем в N статьях. Это шумовой фильтр, но у него есть цена: редкий, но важный термин может исчезнуть.

`flush(state, activity)` — это запись vault. Сначала готовятся имена файлов, затем пишутся заметки источников, концепт-хабы, внешние stubs, хаб-файлы (`_Overview_Synthesis.md`, `_Bibliography.md`, `Formatted_Bibliography.md`, `references.bib`), журнал `_Agent_Activity.md` и `_state.json`.

`ObsidianVaultManager` отвечает за имена и YAML. `clean_name` убирает запрещённые символы, `_truncated_stem` обрезает слишком длинные имена и добавляет хэш, `_register` разрешает коллизии, `_yaml` экранирует значения, чтобы двоеточия и кавычки не ломали frontmatter. Пресет графа Obsidian устанавливается при первом запуске и сливается с пользовательскими настройками, а не перезаписывает их.

Ограничения GraphAgent тоже важны. Каноникализация эвристическая: сложные синонимы могут не слиться, а слишком агрессивное слияние может объединить разные сущности. Graph view — визуализация, а не доказательство качества; QA проверяет ссылки, но не смысл связей.

4.5. EvaluationAgent: независимая проверка

EvaluationAgent — это QA. Его код — `agent/evaluation_agent.py` и `qa/vault_checks.py`. Роль — QA / Evaluation Engineer. Задача: независимо проверить, что vault и состояние не противоречат друг другу.

`audit(state, vault_path)` запускает `run_vault_checks(vault_path)` и `run_state_checks(state)`, объединяет отчёты и логирует summary. Проверки vault смотрят на файлы и ссылки: существует ли папка, есть ли Markdown-файлы, нет ли одинаковых имён, есть ли frontmatter, не висят ли `[[ссылки]]`, не являются ли заметки сиротами, не пуста ли библиография.

Проверки состояния смотрят на данные: есть ли статьи, есть ли summary и citation, есть ли концепты, не ссылается ли статья на отсутствующий hub, есть ли определения у концептов, не пусты ли библиография и обзор, нет ли дубликатов рёбер и не ссылаются ли рёбра на неизвестные статьи.

Severity разделяет проблемы на `error`, `warning` и `info`. `error` критичен, обычно ломает целостность. `warning` подозрителен, но база может работать. `info` нужен для диагностики. `litgraph --check --id ...` печатает отчёт; в CI можно проверять наличие errors.

Ограничение QA в том, что он проверяет структуру и целостность, а не научную истинность. Он не может понять, что статья нерелевантна, если LLM её пропустил. Он не заменяет бенчмарк качества извлечения. Но он ловит те ошибки, которые человек не заметит в базе из сотни заметок: битые ссылки, дубли, отсутствующий frontmatter, сиротские концепты.

5. Сквозные сценарии и данные

5.1. Как тема превращается в vault

Сценарий создания базы начинается с команды CLI. Пользователь передаёт уникальный `--id`, тему, требования, формат цитирования и квоту статей. CLI проверяет аргументы, создаёт или открывает vault, загружает настройки и передаёт управление Lead-агенту.

Lead формирует пустое состояние и запускает LangGraph. Первый узел, `plan_request`, решает, нужен ли LLM для выбора поисковых движков. Если пользователь передал `--engines`, план строится вручную; если `--no-router`, используется эвристика; иначе модели предлагается выбрать primary и fallback из каталога. `finalize_plan` валидирует это решение через `SearchRouter` и сохраняет `search_plan` в состоянии и будущем `_state.json`.

Следующий узел, `search`, обращается к выбранным движкам. Если primary даёт мало результатов, подключается fallback. Кандидаты дедуплицируются по DOI, URL и названию, а каждый кандидат получает `source_engine`. Затем `validate` просит LLM проверить тему и требования. Кандидаты, которые не прошли порог, отбрасываются, а причина остаётся в логах.

Если после валидации статей меньше квоты, `suggest_queries` генерирует подзапросы, и цикл `search → validate` повторяется. Всего допускается до трёх раундов. `seen_ids` не даёт бесконечно возвращать одни и те же статьи. Когда статей достаточно или раунды закончились, `rank_select` сортирует кандидатов по наукометрическому скору и применяет elbow cutoff. В результате остаётся «голова» списка — те статьи, которые действительно стоит обрабатывать.

Дальше начинается ingestion. Для каждой выбранной статьи `expand_citations` подтягивает references и citations и превращает их в рёбра `cites`. Затем `synthesize` извлекает summary, findings, methodology, limitations и концепты. В deep-режиме `fetch_pages` скачивает PDF, а `ground_concepts` привязывает концепты к страницам. После этого `commit_paper` добавляет статью в `base`, и цикл повторяется для следующей статьи.

Когда все статьи обработаны, `rebuild_graph` канонизирует концепты и строит рёбра. `define_concepts` пишет определения концептов батчами. `bibliography` собирает цитаты, `link_network` строит типизированные связи, `overview` пишет общий обзор. `flush` записывает vault, а `audit` проверяет ссылки, frontmatter, дубли, сирот и целостность. В конце пользователь видит таблицу выбранных статей и QA-отчёт.

Эта последовательность важна не только как список шагов. Она показывает, где именно принимаются решения. Поиск — это решение Retrieval и LLM. Отбор — это решение Validator и Ranking. Извлечение — это решение Prompt. Граф — это решение Graph. Проверка — это решение QA. Если результат плохой, можно понять, на каком шаге он стал плохим, вместо того чтобы винить «ИИ вообще».

5.2. Как работает обновление базы

Обновление начинается с `--update`. Lead загружает `_state.json` и передаёт существующую базу в workflow. Тема становится комбинацией исходной темы и директивы, а `queries=[directive]`. Это важно: обновление не должно пересобирать всю базу с нуля и не должно терять старые статьи.

После валидации workflow не делает повторный `rank_select`, потому что новые статьи уже известны. Он сразу идёт в `expand_citations`, затем в синтез и commit. Когда новые статьи добавлены,

`rebuild_graph` перестраивает граф, но `GraphAgent` сохраняет существующие определения концептов. `save_state` записывает обновлённый `_state.json`, а `audit` проверяет итог.

Главная гарантия обновления: старые данные не перезаписываются. Если пользователь отредактировал определение концепта вручную, `--update` не затрёт его. Если новая статья упоминает уже известный концепт, hub обновится, но определение останется. Это делает базу «живой»: её можно расширять неделями, не боясь потерять ручную работу.

5.3. Deep-режим и grounding

Deep-режим включается флагом `--deep`. После синтеза статьи система скачивает open-access PDF и извлекает текст постранично. Затем `ground_concepts` ищет упоминания концептов по страницам и заполняет `ConceptMention`: `concept_name`, `paper_title`, `page_number`, `context_snippet`.

Это нужно, чтобы в концепт-хабе появились ссылки вида «источник, страница, цитата», а не просто список статей. Пользователь может открыть страницу PDF и проверить, действительно ли концепт там упоминается. Это повышает доверие к графу.

Если PDF недоступен, deep-режим деградирует: grounding пропускается, статья всё равно обрабатывается по абстракту. Deep не является обязательным для работы; это дополнительный слой качества.

5.4. Проверка, список и удаление

`--check` запускает QA и печатает issues. `--list` показывает существующие базы. `--delete` удаляет базу; это необратимо, поэтому команда должна использоваться осторожно. `--check` полезен не только перед защитой, но и в повседневной работе: после большого обновления он быстро показывает, не появились ли битые ссылки или сироты.

5.5. Модель данных

Модели лежат в `lit_graph/models.py`. Понимание этих классов важно, потому что именно они описывают, что система считает «статьёй», «концептом» и «связью».

`Paper` — статья. В ней есть идентификация (`id`, `title`, `authors`, `year`, `venue`, `doi_or_url`, `pdf_url`, `source_engine`), наукометрия (`citation_count`, `influential_citation_count`, `citation_velocity`, `influential_ratio`, `objective_scientific_score`, `normalized_weight`, `is_elbow_selected`), валидация (`topic_relevance_score`, `topic_relevance_reason`, `requirements_met`, `requirements_reason`, `scientific_value_score`), синтез (`summary_en`, `key_findings_en`, `methodology_en`, `limitations_en`, `extracted_concepts`, `extracted_concepts_all`, `concept_mentions`), связи (`referenced_paper_titles`, `citing_paper_titles`, `related_paper_titles`) и библиография (`formatted_citation`).

Поле `extracted_concepts` — это усечённый список до `MAX_CONCEPTS_PER_PAPER`. Поле `extracted_concepts_all` — полный список до усечения. Зачем два списка? Потому что каноникализация может отфильтровать часть концептов, а при `--update` нужно иметь возможность пересчитать граф, не потеряв идеи, которые были отфильтрованы раньше. Это пример того, как деталь хранения влияет на качество обновлений.

`ConceptHub` — концепт. В нём есть `name`, `summary_en`, `aggregated_score`, `normalized_weight` и `mentions`. `ConceptMention` — упоминание концепта в статье: `concept_name`, `paper_title`, `page_number`,

`context_snippet`. `GraphEdge` — ребро: `source_title`, `target_title`, `relation_type`. `LiteratureBaseState.add_edge` не добавляет дубликаты, сравнивая нормализованные концы и тип.

`LiteratureBaseState` — вся база: `base_id`, `topic`, `requirements`, `citation_format`, `overall_summary_en`, `papers`, `concepts`, `edges`, `bibliography_en`, `search_plan`, `created_at`, `updated_at`. Это состояние сериализуется в `_state.json` и восстанавливается при следующем запуске.

5.6. `_state.json` как источник правды

`_state.json` — JSON-представление `LiteratureBaseState`. Он лежит в корне vault. В нём есть статьи, концепты, рёбра, библиография, обзор, `search_plan` и временные метки. Пример сокращённо выглядит так:

```
{
  "base_id": "transformer_circuits",
  "topic": "Mechanistic Interpretability...",
  "requirements": "Peer-reviewed...",
  "citation_format": "APA 7th",
  "papers": {
    "2401.00001": {
      "id": "2401.00001",
      "title": "Example Paper",
      "year": 2024,
      "citation_count": 42,
      "extracted_concepts": ["Induction Head", "Transformer"],
      "source_engine": "arxiv"
    }
  },
  "concepts": {
    "Induction Head": {
      "name": "Induction Head",
      "summary_en": "A circuit that...",
      "mentions": []
    }
  },
  "edges": [],
  "search_plan": {
    "primary": ["arxiv", "semantic_scholar"],
    "fallback": ["web"],
    "source": "llm"
  }
}
```

Запись атомарная: сначала создаётся `.json.tmp`, затем файл заменяется. Если процесс упал, старый файл остаётся целым. Это особенно важно для длинных прогонов на локальной модели, которые могут занимать десятки минут.

5.7. Структура vault

Vault — это папка Obsidian. В корне лежат `Formatted_Bibliography.md`, `_Bibliography.md`, `_Overview_Synthesis.md`, `_Agent_Activity.md`, `_state.json` и `references.bib`. В `Sources/` лежат заметки источников и внешние заглушки, в `Concepts/` — концепт-хабы.

Заметка источника начинается с frontmatter. В нём есть `id`, `title`, `authors`, `year`, `venue`, `url`, `pdf_url`, `source_engine`, `citation_count`, `citation_velocity`, `objective_score`, `node_weight` и `type: literature_note`. Дальше идут секции: Formatted Citation, Executive Summary, Key Findings, Methodology & Limitations, Knowledge Graph Connections, Typed Graph Relations (если есть) и Abstract.

Концепт-хаб содержит `concept`, `aggregated_score`, `node_weight` и `type: concept_hub`. Внутри — определение и grounded citations с страницами и цитатами. Внешняя заглушка содержит `title` и `type: external_reference`, а внутри — ссылки на источники, которые её цитируют.

Хаб-файлы собирают базу воедино. `_Overview_Synthesis.md` содержит обзор, индекс и типизированные связи. `_Bibliography.md` — интерактивная библиография с `[[links]]`. `Formatted_Bibliography.md` — чистый список для копирования. `references.bib` — BibTeX. `_Agent_Activity.md` — журнал: кто, что, когда и сколько заняло.

5.8. Почему vault, а не база данных

Vault выбран по трём причинам. Во-первых, прозрачность: каждую заметку можно открыть и поправить руками. Во-вторых, локальность: нет облачной зависимости и vendor lock-in. В-третьих, Obsidian уже даёт граф, обратные ссылки и поиск; не нужно писать собственный UI.

Минус в том, что нет транзакций и строгой схемы. Поэтому в проекте есть QA, детерминированные имена, YAML-экранирование и `_state.json` как источник правды. Если бы мы выбрали базу данных, пришлось бы писать миграции и UI, а пользователь потерял бы возможность править заметки в любимом редакторе.

5.9. Что происходит при повторном запуске

При повторном запуске `_state.json` загружается. Существующие статьи не дублируются по `id`, существующие концепты переиспользуются, определения не перезаписываются, новые статьи добавляются. Граф перестраивается, но пользовательские настройки `graph.json` сохраняются. QA проверяет итог.

Именно это делает базу «живой», а не одноразовым экспортом. Пользователь может запускать обновления по мере появления новых статей, и база будет расти, сохраняя структуру и ручные правки.

6. Как устроены поиск, LLM и граф

6.1. Поиск и выбор движков

Поиск начинается не с запроса, а с каталога. В `search/tools.py` каждый движок описан как данные: ключ, название, краткое описание, сильные и слабые стороны, лучшие сценарии. `catalog()` возвращает этот список для CLI `--tools`, а `catalog_prompt()` превращает его в текст для LLM. Модель видит не «чёрный ящик», а описание: Semantic Scholar хорош для метаданных и цитирований, arXiv — для свежих preprint-ов, open web — для случаев, когда академические API не дают результат.

`SearchRouter` (`search/router.py`) не доверяет ответу модели напрямую. Он проверяет, что ключи известны, нормализует регистр и пробелы, убирает дубликаты, ограничивает число primary, гарантирует непустой fallback и сохраняет источник решения (`llm`, `manual`, `heuristic`). Если модель вернула мусор, router строит эвристический план. Если пользователь передал `--engines`, LLM вообще не вызывается. Если `--no-router`, используется эвристика. Это пример «LLM предлагает, система проверяет»: модель помогает адаптироваться, но не может сломать пайплайн невалидным ответом.

Почему не опросить все движки сразу? Потому что это дорого и шумно. Модель выбирает один-два primary и fallback, исходя из темы. Для свежих preprint-ов лучше arXiv, для цитатной наукометрии — Semantic Scholar, для узких веб-источников — web. Fallback добавляется, если primary дал меньше `SEARCH_FALLBACK_MIN_RESULTS` результатов.

Кандидаты приходят из разных источников, поэтому их нужно дедуплицировать. Сравниваются DOI, URL и нормализованное название. `merge_metadata` не затирает непустые поля пустыми и дополняет отсутствующие. Это важно, потому что arXiv может дать abstract, а Semantic Scholar — citation counts. Если просто перезаписать запись последним ответом, можно потерять полезные метаданные.

6.2. Наукометрический скор

Ранжирование — это попытка ответить на вопрос: какая статья с большей вероятностью будет полезна исследователю? В проекте используется эвристический скор, который объединяет три сигнала: скорость цитирования, объём цитирования и долю влиятельных цитирований.

Формула выглядит так:

```
age = max(1, current_year - year)
velocity = citation_count / age
influential_ratio = (influential_citation_count + 1) / (citation_count + 1)
log_volume = log10(citation_count + 1)

score = min(velocity, 100) * 0.40
      + log_volume * 4.0
      + influential_ratio * 10.0 * 0.25
```

Разберём каждый член. `age` — возраст статьи, но не меньше единицы, чтобы не делить на ноль для статьи текущего года. `velocity` — сколько цитирований в год в среднем. Свежая статья с 20 цитированиями за год может быть полезнее старой статьи с 100 цитированиями за десять лет. `min(velocity, 100)` защищает формулу от выбросов: одна супер-цитируемая статья не должна доминировать над всем списком.

`influential_ratio` — доля влиятельных цитирований. Мы добавляем единицу к числителю и знаменателю, чтобы избежать деления на ноль и слишком резких значений для статей с малым числом

цитирований. Этот сигнал говорит не «сколько раз статью упомянули», а «сколько раз её упомянули как важную».

`log_volume` — логарифм общего числа цитирований. Логарифм нужен потому, что цитирования распределены очень неравномерно: статья с 10 000 цитирований не должна получать в сто раз больший вклад, чем статья с 100. Логарифм сжимает диапазон и делает вклад более умеренным.

Веса 0.40, 4.0 и 10.0 * 0.25 — эвристические. Они не откалиброваны на размеченном наборе, и это честно зафиксировано в отчёте. Смысл не в том, чтобы получить «истинную» наукометрию, а в том, чтобы отсортировать статьи так, чтобы в голове списка оказались работы с хорошим балансом свежести, объёма и качества цитирований.

Пример: статья 2022 года, 120 цитирований, 20 влиятельных. Пусть текущий год — 2026. Тогда `age=4`, `velocity=30`, `influential_ratio=(20+1)/(120+1)=0.174`, `log_volume=log10(121)≈2.083`. Скор:

```
score = min(30, 100) * 0.40 + 2.083 * 4.0 + 0.174 * 10 * 0.25
      = 12.0 + 8.332 + 0.435
      ≈ 20.77
```

Другая статья 2025 года, 10 цитирований, 2 влиятельных: `age=1`, `velocity=10`, `influential_ratio=3/11≈0.273`, `log_volume=log10(11)≈1.041`. Скор ≈ `4.0 + 4.164 + 0.682 = 8.85`. Первая статья выигрывает, потому что у неё больше объём и скорость, несмотря на меньшую долю влиятельных.

Скор не является «истиной». У него есть ограничения: citation counts могут быть неполными, разные области цитируют с разной скоростью, а веса не откалиброваны. Поэтому ranking — это фильтр для отсечения явного шума, а не автоматический выбор «лучших статей всех времён».

6.3. Метод локтя

После сортировки по скору нужно решить, сколько статей оставить. Если оставить всё, в базу попадёт низкоимпактный хвост. Если оставить слишком мало, можно потерять полезные работы. Метод локтя (elbow cutoff) — это эвристика, которая ищет границу между «головой» и «хвостом» распределения.

Идея проста. Если отсортировать скоры по убыванию, в голове они падают медленно, затем где-то происходит резкое падение, а в хвосте снова относительно ровно. Точка резкого падения — «локоть» — и есть граница. Формально алгоритм считает первые разности (drops) и их изменения (curvature):

```
ordered = sorted(scores, reverse=True)
drops[i] = ordered[i] - ordered[i + 1]
curvature[i] = drops[i] - drops[i + 1]
```

Большая положительная curvature в позиции `i` означает, что падение после ранга `i+1` намного резче, чем следующее падение. Значит, голова заканчивается после `i+1` статей. Алгоритм выбирает индекс максимальной положительной curvature, прибавляет единицу и ограничивает результат диапазоном `min_k..max_k`.

Разберём на примере. Пусть скоры равны `[100, 80, 60, 30, 28, 25, 24]`. Первые разности: `[20, 20, 30, 2, 3, 1]`. Вторые разности: `[0, -10, 28, -1, 2]`. Максимальная положительная curvature — 28 на индексе 2. Значит, `optimal_k = 2 + 1 = 3`: оставляем первые три статьи. Логика здорового смысла совпадает: после 60 идёт резкий провал до 30, а дальше хвост почти плоский.

В коде есть несколько защит. Если `n == 0`, возвращается 0. Если `n <= min_k`, возвращается `max_k`. Если все скоры равны, возвращается `max_k`, потому что локтя нет. `max_k` и `min_k` всегда ограничиваются длиной списка. Это защищает от деления на ноль и от выхода за границы.

Метод локтя — эвристика, а не строгая статистика. Он чувствителен к масштабу скоров и к выбросам. На маленьком списке из трёх статей он почти не имеет смысла. На списке с плато он может выбрать границу произвольно. Поэтому в проекте есть флаг `--no-elbow`, который отключает cutoff и оставляет top `max_papers`. Это важно для воспроизводимости: если исследователь хочет сравнить «все статьи» и «голову», он может переключить режим.

6.4. LLM-слой и промпты

LLM-слой начинается с провайдеров. `llm/base.py` задаёт интерфейс `generate`/`generate_json`, а `llm/providers.py` реализует OpenAI-совместимый клиент. Провайдер отвечает за таймауты, повторы, температуру и опциональный кэш. Профили задаются в `.env`: Ollama, DeepSeek, OpenAI, Groq, OpenRouter, vLLM.

Промпты живут в `llm/prompts.py`. Там есть выбор движков, генерация подзапросов, валидация, синтез статьи, определения концептов, grounding по страницам PDF, библиография, связи и обзор. Важно, что `synthesizer.py` использует `paper_synthesis_prompt`, а не дублирует его inline. Раньше дубликат был, и изменения в `prompts.py` не влияли на бенчмарк. После рефакторинга формулировка живёт в одном месте.

JSON-санитайзер — это защитный слой. Локальные модели часто возвращают `<think>...</think>`, markdown-обёртку, комментарии или обрезанный JSON. Санитайзер сначала удаляет reasoning-блоки, затем снимает code fence, затем пробует `json.loads`, затем ищет самый большой валидный объект или массив. Если ничего не вышло, возвращается `{}` и пишется warning. Это не гарантирует правильность фактов, но гарантирует, что пайплайн не упадёт на первой скобке.

Батчинг — это способ не упереться в лимит токенов. `CONCEPT_BATCH_SIZE` задаёт, сколько концептов определять за один запрос, а `BIBLIOGRAPHY_BATCH_SIZE` — сколько источников форматировать за раз. Без батчинга большие базы получали бы половину пустых определений, потому что ответ обрезался бы на середине JSON.

Промпт v2 для концептов — отдельная история. Старый промпт просил JSON с тремя примерами и не задавал количество. Модель выдавала в среднем 5.75 концепта, часто общих. Новый промпт просит 6–8 конкретных концептов, требует, чтобы они были явно упомянуты, покрывает methods/tasks/metrics/materials/ datasets/technical terms, запрещает generic-слова и дубликаты и ставит `extracted_concepts` первым в JSON. Результат — 8.0 концептов в среднем и прирост F1, описанный в разделе 7.

6.5. Граф и каноникализация

Каноникализация — это то, что превращает набор строк в граф. LLM возвращает `Transformer`, `Transformers`, `GNN`, `GNNs`, `Graph Neural Networks (GNNs)`. Без слияния это четыре разных узла, и граф теряет смысл. `concepts.py` решает эту задачу эвристически.

Сначала `_normalize_whitespace` убирает лишние пробелы, а `_strip_edges` обрезает пунктуацию по краям. Затем `_strip_parenthetical` снимает parenthetical-часть, если она является сокращением: `Graph Neural Networks (GNNs)` превращается в `Graph Neural Networks`, а не в `GNNs`. После этого `concept_key` приводит строку к нижнему регистру, раскрывает известные акронимы из `ABBREVIATIONS` и singularizes

последнее слово с учётом uncountable-слов. `display_name` делает человекочитаемое имя: smart title case, сохранение акронимов и mixed-case.

Пример: `GNNs` → ключ `graph neural network`, display `Graph Neural Network`. `GNN` → тот же ключ. `Graph Neural Networks (GNNs)` → тот же ключ. `choose_display_names` выбирает самый частый display-вариант, а `dedupe_concepts` убирает дубли. Это не идеальная семантика, но для большинства случаев работает.

Вес концепта зависит от числа упоминаний и веса статей. Нормализация в 0..1 позволяет использовать размер узла в graph view. `MIN_CONCEPT_PAPERS` отсекает концепты, которые встретились меньше чем в N статьях. Это снижает шум, но может потерять редкий, но важный термин.

Запись vault идёт через `ObsidianVaultManager`. `write_paper_note` формирует frontmatter, citation, summary, findings, methodology, limitations, связи, typed relations и abstract. `write_concept_note` формирует frontmatter, определение и grounded citations. `write_external_stubs` создаёт лёгкие заметки для внешних ссылок. `write_hub_files` пишет обзор, библиографию, BibTeX и журнал активности.

Имена и YAML — это не мелочи. `clean_name` убирает запрещённые символы, `_truncated_stem` обрезает длинные имена и добавляет хэш, `_register` разрешает коллизии, `_yaml` экранирует значения. Без этого заметка с двоеточием в названии ломала бы frontmatter, а две статьи с одинаковым названием перезаписывали бы друг друга.

Ограничения графа честные. Каноникализация эвристическая: сложные синонимы могут не слиться, а слишком агрессивное слияние может объединить разные сущности. Graph view красив, но не доказывает семантическую правильность связей. Именно поэтому QA и бенчмарк существуют отдельно от «выглядит красиво».

7. QA, тесты и бенчмарк

7.1. Зачем нужен отдельный QA-агент

LLM и внешние API ошибаются. Если писать vault без проверки, можно получить битые ссылки, заметки без frontmatter, дубликаты имён, статьи, которые ссылаются на отсутствующие концепты, пустую библиографию и «сиротские» заметки. Эти ошибки не всегда видны на глаз, особенно когда в базе десятки файлов. Поэтому в проекте есть независимый QA-агент.

QA не мутирует данные. Он читает vault и состояние, собирает issues с severity и отдаёт отчёт. Это позволяет использовать `litgraph --check` в CI и на демонстрации, а также быстро проверять базу после больших обновлений. Независимость важна: если бы QA был частью GraphAgent, он мог бы «подгонять» проверки под то, что только что записал, и не замечать собственные ошибки.

7.2. Проверки vault

`run_vault_checks(vault_path)` проходит по всем `.md` вне `.obsidian` и `.trash`. Сначала он проверяет, что папка существует и что в ней есть Markdown-файлы. Затем собирает имена заметок и ищет дубликаты: одинаковые stem-имена могут привести к тому, что Obsidian не поймёт, куда ведёт ссылка.

После этого каждая заметка читается, и QA смотрит на frontmatter. Если frontmatter нет, это warning. Если тип известен, проверяются обязательные ключи: у `literature_note` должны быть `title` и `type`, у `concept_hub` — `concept` и `type`, у `external_reference` — `title` и `type`, у `synthesis_hub` — `topic` и `type`, у `master_bibliography` — `citation_format` и `type`, у `agent_activity` — `type`. Если тип неизвестен, это тоже warning.

Ссылки проверяются через `LINK_PATTERN`. Для каждой `[[ссылки]]` QA пытается найти файл: если ссылка содержит путь, проверяется путь; если нет — ищется по stem-имени. Ненайденная ссылка попадает в `unresolved_link`. Отдельно считаются входящие и исходящие ссылки, чтобы найти сирот: заметки в `Sources/` и `Concepts/`, у которых нет ни входящих, ни исходящих ссылок. Наконец, проверяется `_Bibliography.md`: если там стоит «No references available», это warning.

Статистика отчёта включает число заметок, источников, концептов, внешних ссылок, ссылок, unresolved links и orphans. Эти числа полезны не только для отладки, но и для отчёта: они показывают, насколько база связная.

7.3. Проверки состояния

`run_state_checks(state)` смотрит не на файлы, а на данные. Если статей нет, это warning. Для каждой статьи проверяется, есть ли summary, formatted citation и концепты. Если статья ссылается на концепт, которого нет в `state.concepts`, это error `orphan_concept_link`: в vault такая ссылка повиснет. Для каждого концепта проверяется, есть ли определение и упоминания. Если библиография или обзор пусты, это warning. Для рёбер проверяются дубликаты и неизвестные концы: source должен быть известной статьёй, target — известной статьёй или внешней ссылкой.

Severity разделяет проблемы на `error`, `warning` и `info`. `error` критичен, обычно ломает целостность. `warning` подозрителен, но база может работать. `info` нужен для диагностики. `EvaluationAgent.audit` объединяет vault- и state-отчёты и логирует summary.

Ограничение QA в том, что он проверяет структуру и целостность, а не научную истинность. Он не может понять, что статья нерелевантна, если LLM её пропустил. Он не заменяет бенчмарк качества извлечения. Но он ловит те ошибки, которые человек не заметит в базе из сотни заметок.

7.4. Тесты и CI

В проекте 55 тестов. Они покрывают pipeline и агентов, персистентность `_state.json`, каноникализацию концептов, имена заметок, QA, батчинг, провайдеры и JSON-санитайзер, поисковый роутер, ранжирование и бенчмарк. Тесты запускаются командой `pytest -q`, линтер — `ruff check lit_graph tests benchmarks`.

CI в GitHub Actions проверяет Python 3.10, 3.11, 3.12 и 3.13. `ruff` валит сборку при ошибках, `pytest` запускается на push и pull request. CI не делает реальных сетевых вызовов и не запускает локальную LLM: E2E проверяется локально, а CI ловит регрессии структуры и логики.

Такое разделение важно. Если бы CI запускал реальную модель, он был бы медленным, дорогим и нестабильным. Поэтому CI проверяет то, что можно проверить детерминированно: импорты, модели, каноникализацию, QA, JSON- санитайзер, роутер и скоринг. А бенчмарк и E2E — это отдельные локальные прогоны.

7.5. SciERC: что это и почему он релевантен

SciERC (Luan et al., EMNLP 2018) — стандартный бенчмарк для scientific knowledge-graph construction. В нём 500 абстрактов научных статей с аннотациями сущностей шести типов (Task, Method, Metric, Material, Generic, OtherScientificTerm), типизированных отношений и coreference-кластеров.

Это не «граф вообще», а именно извлечение научных сущностей и связей из текста. Поэтому SciERC релевантен LitGraphAgent, который тоже строит граф из научных статей. Но есть важное отличие: LitGraphAgent извлекает канонические концепты, а SciERC размечает спаны. Наш пайплайн не предсказывает типы отношений SciERC, поэтому мы не считаем relation F1. Это не «забыли», а сознательное ограничение: считать метрику на данных, которых система не производит, было бы фикцией.

7.6. Что именно измеряет harness

`benchmarks/scierc_benchmark.py` загружает `.txt` и `.ann`, выбирает `limit` абстрактов с `seed`, вызывает production `PromptAgent.synthesize_paper` и сравнивает `extracted_concepts` с gold-упоминаниями. Он считает concept precision/recall/F1 и paper→concept edges, сохраняет JSON и JSONL checkpoint, умеет resume после прерывания и пишет failures, не роняя весь прогон.

Harness не измеряет типизированные отношения, coreference и span boundaries. Он также не оценивает качество итогового обзора или поиска. Это regression sample: 20 абстрактов достаточно, чтобы ловить регрессии, но недостаточно для статистической значимости.

7.7. Exact, relaxed и content-only

Exact — это нормализованное равенство строк: lowercase, singularization, удаление пунктуации и артиклей. Relaxed — это token Jaccard ≥ 0.5. Он допускает модификаторы и сокращения: `word sense disambiguation (WSD) systems` и `word sense disambiguation` считаются совпадением. Content-only —

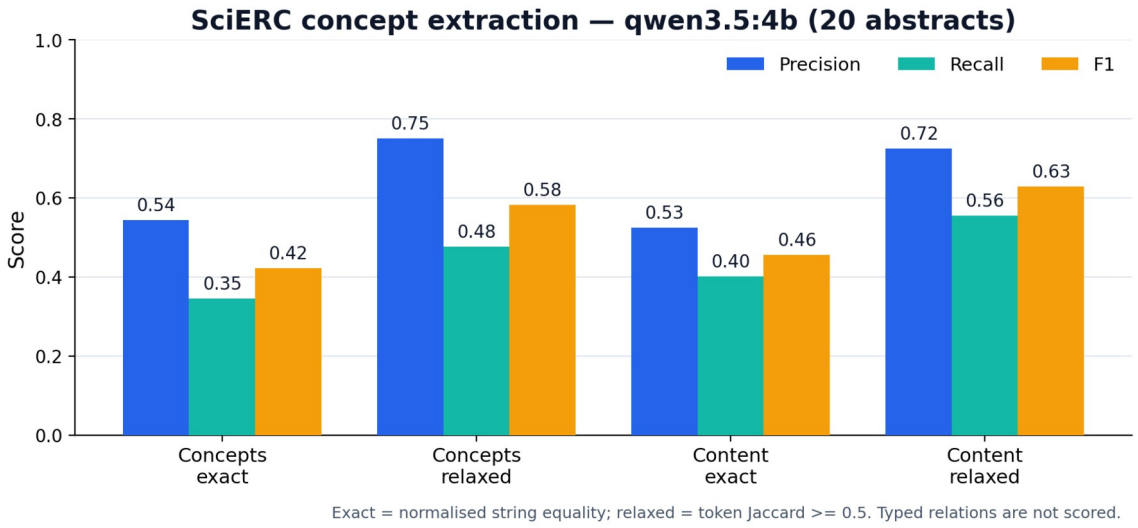
это gold-сущности без типа Generic. Это отдельный сигнал: Generic-слова (`system`, `method`, `approach`) часто шумят, и без них метрика показывает содержательные концепты.

Метрики считаются micro: TP/FP/FN агрегируются по всем абстрактам, затем считаются P/R/F1. Это стандартно для extraction, но нужно явно указывать. Micro доминируется большими статьями, macro — маленькими; в отчёте используется micro.

7.8. Результаты на 20 абстрактах

Протокол: 20 абстрактов, `seed=42`, локальные модели через Ollama, production-вызов `PromptAgent.synthesize_paper`, 0 падений.

Модель	Сигнал	Precision	Recall	F1
`qwen2.5:3b`	Concepts, exact	0.538	0.254	0.345
`qwen2.5:3b`	Concepts, relaxed	0.723	0.341	0.464
`qwen2.5:3b`	Content, relaxed	0.706	0.402	0.512
`qwen3.5:4b`, prompt v1	Concepts, exact	0.556	0.254	0.349
`qwen3.5:4b`, prompt v1	Concepts, relaxed	0.800	0.365	0.501
`qwen3.5:4b`, prompt v1	Content, relaxed	0.774	0.426	0.549
`qwen3.5:4b`, prompt v2	Concepts, exact	0.544	0.345	0.422
`qwen3.5:4b`, prompt v2	Concepts, relaxed	0.750	0.476	0.583
`qwen3.5:4b`, prompt v2	Content, relaxed	0.725	0.555	0.629



Метрики SciERC для qwen3.5:4b, prompt v2

Среднее число концептов: `qwen2.5:3b` — 5.95, `qwen3.5:4b` prompt v1 — 5.75, `qwen3.5:4b` prompt v2 — 8.0, gold — 12.6. Время: `qwen2.5:3b` — 252.7 с, `qwen3.5:4b` prompt v1 — 1836.9 с, `qwen3.5:4b` prompt v2 — 3571.0 с (около 179 секунд на абстракт).

7.9. Что дал prompt v2

Старый промпт не задавал количество концептов и показывал три примера. Модель выдавала около 5.75 концепта, часто общих. Новый промпт просит 6–8 концептов, требует, чтобы они были явно упомянуты, покрывает methods/tasks/metrics/materials/datasets/technical terms, запрещает generic и дубликаты и ставит `extracted_concepts` первым в JSON.

Прирост: exact F1 +0.073, relaxed F1 +0.082, content relaxed F1 +0.080. Цена: почти вдвое дольше, потому что модель генерирует больше концептов и отвечает стабильно длиннее. Для исследовательской базы это приемлемый trade-off; для интерактивного сценария — нет.

Почему prompt v2 сработал? Причин три. Во-первых, модель получила явное количество: 6–8 вместо неопределённого «сколько получится». Во-вторых, концепты идут первыми в JSON, поэтому модель не тратит бюджет на summary перед ними. В-третьих, правила запрещают generic-слова и требуют явной поддержки в абстракте, что снижает шум. Это не «магия промпта», а структурирование задачи.

7.10. Опорные точки

Чтобы числа не висели в вакууме, в отчёте фиксируются опорные точки. Пустое предсказание даёт F1 0.000 — это нижняя граница. Gold oracle даёт 1.000 — это верхняя граница. `qwen2.5:3b` — меньшая модель на тех же 20 абстрактах. `qwen3.5:4b` prompt v1 — предыдущая формулировка. `qwen3.5:4b` prompt v2 — текущий результат.

Точка	Exact F1	Relaxed F1	Content relaxed F1	Смысл
Пустое предсказание	0.000	0.000	0.000	нижняя граница
`qwen2.5:3b`	0.345	0.464	0.512	меньшая модель
`qwen3.5:4b`, prompt v1	0.349	0.501	0.549	предыдущий промпт
`qwen3.5:4b`, prompt v2	0.422	0.583	0.629	текущий результат
Gold oracle	1.000	1.000	1.000	верхняя граница

Внешний ориентир — SciERC SOTA (SURE, COLING 2025): entity F1 73.4, relation F1 56.9, strict relation F1 44.3. Это span-level NER/RE, а не извлечение канонических концептов. Прямое сравнение некорректно: у нас другая постановка и другой матчинг. Поэтому SOTA даётся как контекст, а не как лидерборд.

7.11. Ограничения бенчмарка

Ограничения нужно понимать, чтобы не переоценить результат. 20 абстрактов — мало для статистической значимости. Один seed. Одна модель, плюс меньшая 3B для сравнения. Оценены только концепты, не отношения. Нет end-to-end метрики поиска (precision@k, nDCG). Нет оценки качества итогового обзора. LLM недетерминирована; один прогон не равен среднему. Exact — нижняя граница из-за каноникализации. SOTA SciERC — другая задача.

До КП3 имеет смысл расширить выборку до 100 абстрактов, добавить 3 seed, 2–3 модели, triple-level F1 (если появятся типы отношений), coverage, orphan rate, unresolved link rate, precision@k/nDCG для поиска, p95 latency и cost per run. Это не «для галочки»: каждое расширение закрывает конкретную слепую зону текущей оценки.

7.12. Как читать эту таблицу

Смотрите на content relaxed F1 как на основной сигнал качества концептов. Смотрите на exact F1 как на нижнюю границу: каноникализация мешает точному совпадению. Сравнивайте только строки с одинаковым протоколом (модель, seed, prompt). Не сравнивайте наш concept F1 с SciERC entity/relation F1 напрямую. Помните, что 20 абстрактов — regression sample, а не лидерборд. Если вы забудете эти правила, таблица начнёт «врать» сама по себе.

8. Метрики с нуля

Этот раздел можно читать отдельно от проекта. Он объясняет, что означают числа в таблицах, почему нельзя сравнивать метрики разных задач и как интерпретировать результаты, не обманывая себя.

8.1. Задача извлечения концептов

У нас есть абстракт статьи. Есть gold — эталонный список концептов, размеченный людьми. Есть prediction — список, который вернула система. Нужно ответить на два вопроса: насколько найденное правильное и насколько полно найдено эталонное. Первый вопрос измеряется precision, второй — recall. F1 сводит их в одно число, но не заменяет оба.

В отличие от классификации, здесь нет «правильного отрицания». Пространство всех возможных концептов огромно, и утверждение «система правильно не назвала случайное слово» не имеет смысла. Поэтому мы не считаем accuracy и не используем TN. Мы работаем с TP, FP и FN.

TP (true positive) — концепт есть в gold и система его назвала. FP (false positive) — система назвала концепт, которого в gold нет. FN (false negative) — концепт есть в gold, но система его пропустила. Эти три числа — основа всех дальнейших метрик.

8.2. Precision

Precision отвечает на вопрос: «Из всего, что система назвала, какая доля действительно правильная?» Формула:

```
Precision = TP / (TP + FP)
```

Пример: система вернула 8 концептов, 5 из них есть в gold. Тогда `Precision = 5 / 8 = 0.625`. Precision падает, когда система добавляет лишние, шумные концепты. Если система вернула 100 концептов, из которых правильные только 5, precision будет 0.05, даже если recall высокий.

Precision особенно важна в графе знаний: лишний концепт создаёт лишний узел, лишние связи и шум в graph view. Пользователь видит не «100 идей», а «100 узлов, половина из которых мусор».

8.3. Recall

Recall отвечает на вопрос: «Из всего, что было в gold, какую долю система нашла?» Формула:

```
Recall = TP / (TP + FN)
```

В gold 10 концептов, найдено 5. Тогда `Recall = 5 / 10 = 0.5`. Recall падает, когда система пропускает концепты. В нашей задаче recall часто ниже precision, потому что модель возвращает 6–8 концептов, а gold содержит в среднем 12.6. Это ограничение бюджета концептов, а не обязательно «глупость» модели.

Recall важен, потому что пропущенный концепт — это потерянная связь. Если статья про induction heads, а концепт `Induction Head` не извлечён, граф теряет ключевой узел, и пользователь не найдёт статью по этому концепту.

8.4. F1

F1 — гармоническое среднее precision и recall:

```
F1 = 2 * Precision * Recall / (Precision + Recall)
```

F1 сильнее наказывает за дисбаланс, чем обычное среднее. Если precision 1.0, а recall 0.1, обычное среднее было бы 0.55, а F1 — около 0.18. Это полезно, когда нужно одно число, но опасно, если забыть, что именно оно скрывает.

Пример: `P = 0.625`, `R = 0.5`. Тогда `F1 = 2 * 0.625 * 0.5 / (0.625 + 0.5) = 0.625 / 1.125 ≈ 0.556`. Это число означает «баланс неплохой, но есть куда расти и по precision, и по recall».

F1 удобен для сравнения моделей, но он не говорит, что именно улучшать. Можно поднять F1, вернув больше концептов (recall растёт, precision падает), или подняв точность (precision растёт, recall падает). Поэтому в отчёте всегда есть и P, и R, и F1.

8.5. Почему не accuracy

Accuracy = (TP + TN) / все. В извлечении концептов TN огромно и не имеет смысла: «правильно не назвать случайное слово» — не полезная работа. Если бы мы считали accuracy, модель, которая всегда возвращает пустой список, могла бы получить высокий процент «правильных отрицаний» и выглядеть лучше, чем она есть. Поэтому accuracy здесь не используют.

8.6. Exact и relaxed

Exact — это точное совпадение после нормализации. Нормализация включает lowercase, единственное число, удаление пунктуации и артиклей. Пример:

```
gold:       "word sense disambiguation (WSD) systems"
prediction: "word sense disambiguation"
exact:      нет (строки разные)
```

Relaxed — это token Jaccard:

```
Jaccard = |пересечение токенов| / |объединение токенов|
```

Если Jaccard ≥ 0.5, считаем совпадением. В примере выше пересечение `{word, sense, disambiguation}` и объединение `{word, sense, disambiguation, wsd, system}` дают Jaccard 3/5 = 0.6, поэтому relaxed считает это совпадением.

Exact всегда ниже relaxed, потому что каноникализация меняет строки. Exact — нижняя граница: если даже строки совпали, это точно хороший сигнал. Relaxed — более честная оценка смысла, но она может склеить близкие, но разные концепты. Поэтому в отчёте есть оба числа.

8.7. Micro и macro

Micro — складываем TP/FP/FN по всем примерам и считаем P/R/F1 один раз. Доминируют большие статьи. Macro — считаем P/R/F1 для каждой статьи, затем усредняем. Все статьи равны, даже маленькие.

Пример: статья A: TP=5, FP=3, FN=5; статья B: TP=1, FP=1, FN=9. Micro: TP=6, FP=4, FN=14, P=6/10=0.6, R=6/20=0.3, F1=0.4. Macro: A: P=0.625, R=0.5, F1=0.556; B: P=0.5, R=0.1, F1=0.167; macro F1=(0.556+0.167)/2=0.361. Числа отличаются, поэтому в отчёте важно писать, какой режим используется. В нашем бенчмарке используется micro.

8.8. Content-only

Generic-сущности (`system`, `method`, `approach`, `algorithm`) часто являются шумом. Content-only исключает их из gold и считает метрику по содержательным концептам. Это не «улучшение модели», а другой срез оценки. Если модель предсказывает generic-слова, в content-only они становятся FP и снижают precision. Поэтому content-only дисциплинирует: она показывает, насколько система извлекает именно содержательные идеи, а не общие слова.

8.9. MRR

MRR (Mean Reciprocal Rank) используется в задачах ранжирования. Для каждого запроса правильный ответ имеет ранг; берём `1/rank` и усредняем:

```
MRR = (1 / |Q|) * Σ (1 / rank_i)
```

Пример: правильный ответ на позициях 1, 2 и 4. Тогда `MRR = (1/1 + 1/2 + 1/4) / 3 = (1 + 0.5 + 0.25) / 3 ≈ 0.583`. MRR хорош, когда важен первый правильный ответ. Он сильно вознаграждает ранг 1 и почти не чувствует разницу между 100 и 101.

8.10. Hits@k

Hits@k — доля запросов, где правильный ответ попал в top-k:

```
Hits@k = число запросов с rank ≤ k / все запросы
```

Пример: 100 запросов, в 60 правильный ответ в top-10 → `Hits@10 = 0.6`. Hits@k не учитывает, на каком именно месте внутри top-k; для этого есть MRR и nDCG.

8.11. nDCG

nDCG учитывает и порядок, и градацию релевантности:

```
DCG@k = Σ (2^rel_i - 1) / log2(i + 1)
nDCG@k = DCG@k / IDCG@k
```

`IDCG` — DCG идеального порядка. nDCG ∈ [0,1]. Пример: релевантности `[3, 2, 0, 1]`.

```
DCG = (2^3-1)/log2(2) + (2^2-1)/log2(3) + 0 + (2^1-1)/log2(5)
    = 7/1 + 3/1.585 + 1/2.322
    ≈ 7 + 1.893 + 0.431 = 9.324
```

Идеальный порядок `[3,2,1,0]` даст IDCG; nDCG = DCG/IDCG. Если nDCG близок к 1, порядок почти идеальный; если близок к 0, релевантные документы оказались внизу.

8.12. MAP

Average Precision (AP) — площадь под precision-recall кривой для одного запроса. MAP — среднее AP по запросам. MAP полезен в retrieval, когда важна не только первая находка, но и весь список. Если релевантные документы идут подряд в начале, AP высокий; если они разбросаны, AP ниже.

8.13. Triple F1

Для графа знаний важно не только найти узлы, но и правильно соединить их. Предсказанная тройка `(h, r, t)` считается правильной, если после entity linking и каноникализации она совпала с gold. Тогда:

```
Triple P = correct triples / predicted triples
Triple R = correct triples / gold triples
Triple F1 = 2PR / (P + R)
```

Это самая честная метрика «граф построен правильно», но она требует размеченных отношений. В нашем бенчмарке её нет, потому что пайплайн не предсказывает типы отношений SciERC. Если в будущем появятся типы, triple F1 станет главным кандидатом на основную метрику.

8.14. Coverage, orphan rate и unresolved link rate

Для vault полезны инженерные метрики. Node coverage — доля gold-концептов, попавших в граф. Edge coverage — доля gold-связей. Orphan rate — доля заметок без ссылок. Unresolved link rate — доля `[[ссылок]]`, не находящих цель. Schema conformance — доля заметок с правильным frontmatter. Graph density — `2E / (N(N-1))` для неориентированного графа. Connected components — число компонент связности.

Эти метрики не заменяют P/R/F1, но показывают, можно ли пользоваться базой. Можно иметь высокий concept F1 и при этом кучу битых ссылок; пользователь такой базой не сможет нормально пользоваться. Поэтому QA и инженерные метрики нужны вместе с качеством извлечения.

8.15. Статистическая честность

Один прогон на одном seed — это не среднее. LLM недетерминирована, а выборка из 20 абстрактов мала. Поэтому корректный отчёт должен указывать seed, число прогонов, модель и версию данных. Если есть несколько seed, нужно приводить mean ± std, а не лучший результат. Если есть несколько моделей, нужно сравнивать их на одном протоколе.

В нашем отчёте это ограничение зафиксировано: 20 абстрактов, один seed, один прогон на модель. Это regression sample, а не лидерборд. До КП3 планируется 100 абстрактов, 3 seed и 2–3 модели.

8.16. Как читать таблицу LitGraphAgent

Сначала смотрите на content relaxed F1 как на основной сигнал качества концептов. Затем на exact F1 как на нижнюю границу: каноникализация мешает точному совпадению. Сравнивайте только строки с одинаковым протоколом (модель, seed, prompt). Не сравнивайте наш concept F1 с SciERC entity/relation F1 напрямую. Помните, что 20 абстрактов — regression sample, а не лидерборд.

Если вы видите, что exact F1 вырос, а relaxed упал, это может означать, что модель стала точнее в строках, но потеряла смысловые совпадения. Если recall вырос, а precision упал, модель вернула

больше концептов, но с шумом. Если F1 вырос, но время выросло вдвое, это trade-off, а не бесплатный прогресс.

8.17. Частые ошибки

Первая ошибка — сравнивать exact F1 с relaxed F1 и делать вывод «модель стала хуже». Это разные метрики. Вторая — сравнивать concept F1 с SciERC entity F1. Это разные задачи. Третья — считать relation F1, если типы отношений не предсказываются. Четвёртая — показывать лучший seed и молчать про остальные. Пятая — забывать про micro/macro. Шестая — оптимизировать метрику, а не качество графа: например, вернуть 100 концептов, чтобы поднять recall, и разрушить precision. Седьмая — считать 20 абстрактов «лидербордом».

8.18. Главное

Метрика — это линза, а не истина. Одно число не описывает систему. Поэтому в проекте есть несколько сигналов: exact, relaxed, content-only, число концептов, failures, время, QA и тесты. Только вместе они дают честную картину. Если вы научитесь читать эти сигналы вместе, вы сможете обсуждать качество проекта не на уровне «мне кажется, стало лучше», а на уровне конкретных чисел и ограничений.

9. Запуск, код и команда

9.1. Установка

Проект ставится как обычный Python-пакет. Сначала клонируйте репозиторий, затем создайте виртуальное окружение и установите зависимости в режиме разработки. На Windows активация окружения выглядит как `.venv\Scripts\activate`, на macOS и Linux — `source .venv/bin/activate`. Команда `pip install -e ".[dev]"` ставит и сам пакет, и инструменты разработки: pytest, ruff и другие.

`pypdf` входит в основные зависимости, потому что он нужен для deep-режима. Если вы не планируете скачивать PDF, это всё равно не лишняя зависимость: deep-режим должен работать «из коробки», а не требовать отдельной установки.

9.2. Настройка `.env`

Все настройки читаются из `.env` в корне проекта. Для локальной Ollama типичный конфиг выглядит так:

```
LLM_API_KEY=ollama
LLM_BASE_URL=http://localhost:11434/v1
LLM_MODEL=qwen3.5:4b

DEFAULT_VAULTS_DIR=./vaults
MAX_PAPERS_PER_RUN=5
MIN_PAPERS_PER_RUN=2
MAX_SEARCH_CANDIDATES=15
CANDIDATE_EVAL_LIMIT=8
CITATION_EXPANSION_LIMIT=2
ENABLE_ELBOW_CUTOFF=true

SEARCH_ROUTER_ENABLED=true
SEARCH_ROUTER_MAX_PRIMARY=2
SEARCH_FALLBACK_MIN_RESULTS=5

MIN_CONCEPT_PAPERS=2
MAX_CONCEPTS_PER_PAPER=8
CONCEPT_BATCH_SIZE=12
BIBLIOGRAPHY_BATCH_SIZE=20

CACHE_ENABLED=true
CACHE_TTL_HOURS=168
```

Для облачной модели достаточно поменять три переменные: `LLM_API_KEY`, `LLM_BASE_URL` и `LLM_MODEL`. Например, для DeepSeek это `sk-...`, `https://api.deepseek.com` и `deepseek-chat`. Для OpenRouter, Groq, OpenAI или vLLM — соответственно их endpoint и модель. Главное, чтобы endpoint был OpenAI-совместимым.

Не коммитьте `.env` с реальным ключом. В репозитории есть `.env.example`, который показывает структуру без секретов. Если ключ случайно попал в git, его нужно отозвать и переписать историю; это не та ошибка, которую можно «замазать» следующим коммитом.

9.3. Проверка окружения

Перед первым запуском полезно проверить три команды. `litgraph --version` показывает версию. `litgraph --team` показывает пять ролей и владельцев. `litgraph --tools` показывает каталог поисковых движков, из которого выбирает модель. Если `--tools` падает, значит, не установились зависимости или сломан импорт; если `--team` показывает не пять ролей, значит, вы запускаете не ту версию пакета.

9.4. Основные команды

Создание базы — главная команда. Она принимает `--id`, `--topic`, `--requirements`, `--format` и `--max-papers`. `--id` задаёт имя папки vault, `--topic` — тему, `--requirements` — ограничения (например, «peer-reviewed after 2021»), `--format` — формат цитирования, `--max-papers` — квоту. Если база с таким `--id` уже существует, CLI попросит использовать `--update`.

Обновление базы принимает `--update` с директивой. Это может быть новая тема, уточнение или запрос на добавление конкретных работ. `--max-papers` задаёт, сколько новых статей добавить. Старые статьи и определения не перезаписываются.

Deep-режим включается флагом `--deep` при create или update. Он скачивает open-access PDF и привязывает концепты к страницам. Если PDF нет, deep деградирует, но не ломает запуск.

Список, проверка и удаление — служебные команды. `--list` показывает базы, `--check --id ...` запускает QA, `--delete --id ...` удаляет базу. Удаление необратимо, поэтому его стоит использовать только после бэкапа.

Флаги `--engines` и `--no-router` управляют выбором движков. `--engines arxiv,web` заставляет использовать конкретные движки, `--no-router` отключает LLM-роутер и включает эвристику. `--no-cache` отключает дисковый кэш, `--no-elbow` отключает метод локтя, `--verbose` включает debug-логи.

9.5. Тесты и линтер

Перед коммитом полезно запустить `pytest -q` и `ruff check lit_graph tests benchmarks`. Ожидаемо: 55 passed и «All checks passed». Если тесты падают, не коммитьте; если падает ruff, исправьте стиль или добавьте обоснованное исключение. CI всё равно запустит эти команды на Python 3.10–3.13, поэтому локальная проверка экономит время.

9.6. Бенчмарк

Бенчмарк требует SciERC. Скачайте `sciERC_raw.tar.gz` с сайта SciERC, распакуйте и передайте путь к `raw_data` в `--data-dir`. Затем запустите `benchmarks/scierc_benchmark.py` с `--limit`, `--seed`, `--model`, `--output` и `--checkpoint`. Бенчмарк умеет resume: если прервать, повторный запуск продолжит с checkpoint. Failures записываются в JSON, а не роняют весь прогон.

После прогона постройте график командой `python benchmarks/plot_results.py <result.json> --output <png>`. Скрипт читает JSON и рисует precision/recall/F1 по четырём сигналам. График можно вставить в отчёт или презентацию.

9.7. Типичные проблемы

Самая частая проблема — модели нет в реестре Ollama. Например, `qwen3.5:3b` не существует; есть `qwen3.5:4b`. Если вы видите `pull model manifest: file does not exist`, проверьте `ollama list` и имя модели.

Вторая проблема — модель очень медленная. Локальная 4B на слабом GPU может давать 1–2 токена в секунду. Это нормально для ноутбука. Увеличьте `--timeout`, закройте другие GPU-процессы, уменьшите `--limit` для теста. Если нужно быстро проверить пайплайн, используйте `qwen2.5:3b`, но помните, что качество будет ниже.

Третья проблема — таймауты. Поднимите `--timeout` и `--retries`. Для больших баз используйте checkpoint, чтобы не начинать заново. Если сеть нестабильна, включите кэш и повторы.

Четвёртая — JSON-ошибки. Санитайзер восстанавливает большинство случаев, но если модель стабильно ломает JSON, попробуйте другую модель или уменьшите батч. Пятая — нет PDF в deep-режиме. Не все статьи open-access; deep деградирует, и это нормально. Шестая — битые ссылки в Obsidian. Запустите `litgraph --check --id ...`; QA покажет unresolved links, orphans и frontmatter issues.

Седьмая — кириллица в PowerShell. Если вывод Python ломается, задайте `$env:PYTHONIOENCODING='utf-8'`. Это не проблема проекта, а особенность консоли.

9.8. Карта кода

`lit_graph/agent/core.py` — Lead и жизненный цикл. `workflow.py` — LangGraph, узлы и рёбра. `graph_state.py` — `ResearchState`, events и reducer. `roles.py` — роли и владельцы. `retrieval_agent.py` — поиск, citations и PDF. `search/` — клиенты, каталог, router, ranking и PDF. `prompt_agent.py` — LLM-операции. `validator.py` — валидация кандидатов. `synthesizer.py` — синтез, определения, библиография и обзор. `llm/prompts.py` — все промпты. `llm/providers.py` — провайдеры. `graph_agent.py` — онтология, rebuild и flush. `concepts.py` — каноникализация. `storage/obsidian_vault.py` — запись заметок и state. `storage/base_manager.py` — жизненный цикл баз. `evaluation_agent.py` — QA-агент. `qa/vault_checks.py` — проверки. `models.py` — модели данных. `cli.py` — CLI. `config.py` — настройки. `tests/` — 55 тестов. `benchmarks/` — SciERC harness и результаты.

9.9. Как добавить нового агента

Новый агент добавляется в пять шагов. Сначала роль в `agent/roles.py`: ключ, title, owner, handle, responsibility, agent, modules. Затем сервис-класс с обычными методами, который не знает о LangGraph. Затем инстанцирование в `agent/core.py`. Затем узлы и рёбра в `agent/workflow.py`. И наконец тест, обновление `CONTRIBUTORS.md`, `docs/AGENT_TEAM.md`, `README.md` и `.github/CODEOWNERS`.

Инварианты те же: агент не пишет данные другого агента напрямую; все промпты — в `llm/prompts.py`; все записи vault — через `GraphAgent`; QA не мутирует состояние. Если новый агент должен писать в vault, он должен делать это через GraphAgent, а не открывать файлы сам.

9.10. Как добавить поисковый движок

Сначала опишите движок в `search/tools.py`: ключ, название, описание, сильные и слабые стороны, лучшие сценарии. Затем добавьте клиент в `search/`. Подключите его в `search/engine.py`. Добавьте в каталог и тесты. Проверьте `litgraph --tools`. Не добавляйте ветвление «если движок X» внутрь агента: каталог должен оставаться данными.

9.11. Как изменить промпт

Найдите функцию в `llm/prompts.py`, измените текст, сохранив требование JSON. Проверьте, что `synthesizer.py` или `validator.py` действительно используют эту функцию. Раньше был дубликат inline-промпта, и изменения не влияли на бенчмарк. Прогоните `pytest` и бенчмарк на 5–10 абстрактах. Обновите метрики и документацию. Не меняйте промпт «на глаз» без A/B: локальная модель может неожиданно ухудшиться.

9.12. Команда и роли

Иван Холодило отвечает за архитектуру, LangGraph, CLI и состояние. Вадим Лялькин — за онтологию графа, `[[wikilinks]]` и Obsidian. Нодар Чигладзе — за arXiv, Semantic Scholar, web, ranking и PDF. Давид Чигладзе — за QA, тесты, CI и бенчмарк. Илья Синко — за промпты, валидацию и синтез. Подробнее: `CONTRIBUTORS.md`, `docs/kp2/CONTRIBUTIONS.md`, `.github/CODEOWNERS`.

9.13. Дорожная карта

6 октября — MVP: поиск, фильтрация, заметки. 5 ноября — модульный поиск и первые метрики. 25 ноября — тесты, стабилизация, калибровка. 15 декабря — финальный релиз, демо и документация. Первые два этапа закрыты. Третий частично: тесты и QA есть, но калибровка ранжирования и end-to-end метрики поиска ещё не завершены.

До финала имеет смысл расширить бенчмарк до 100 абстрактов и 3 seed, добавить 2–3 модели, triple-level F1 (если появятся типы отношений), precision@k/nDCG для поиска, coverage, orphan rate, unresolved link rate, p95 latency и стоимость прогона, а также демо и скриншоты.

9.14. Риски

Главный риск — узкая оценка: 20 абстрактов, один seed, только концепты. Это честно зафиксировано, но до КП3 нужно расширить выборку. Второй риск — нет relation F1, потому что пайплайн не предсказывает типы; это ограничение архитектуры, а не забытая метрика. Третий — локальная модель медленная и недетерминированная; таймауты, батчинг и меньший `--limit` помогают, но не устраняют проблему. Четвёртый — внешние API могут быть недоступны; retry, cache и fallback смягчают, но не гарантируют результат. Пятый — каноникализация ошибается; QA и ручная проверка нужны. Шестой — вклад участников в git; коммиты по зонам и подтверждение помогают, но требуют дисциплины.

9.15. Что считать успехом

Успех — это не «модель выдала красивый граф». Это когда база воспроизводится, QA не находит критичных ошибок, метрики стабильны на нескольких seed, пользователь может открыть vault и найти нужное, а каждый агент можно объяснить и протестировать отдельно. Если эти пять условий выполнены, проект действительно готов к использованию, а не только к демонстрации.

10. Глоссарий, FAQ и приложение

10.1. Глоссарий

Глоссарий удобно держать таблицей: термин и краткое объяснение. Если термин уже разбирался в предыдущих разделах, здесь даётся сжатая формулировка для быстрого возврата.

Термин	Что это значит
LLM	Большая языковая модель; вероятностный генератор текста
Промпт	Инструкция модели: роль, данные, правила, формат ответа
Токен	Единица текста для модели: слово, часть слова или знак
Контекст	Весь текст, который модель видит в одном запросе
Температура	Параметр случайности генерации
JSON	Текстовый формат структурированных данных
Ollama	Локальный сервер моделей с OpenAI-совместимым API
Провайдер LLM	Сервис или сервер, к которому обращается проект
Агент	Компонент с ограниченной ролью
Оркестрация	Управление порядком вызовов агентов
LangGraph	Библиотека для описания workflow как графа
State	Данные, передаваемые между узлами
Reducer	Функция объединения старого и нового значения поля
Checkpoint	Сохранённая точка состояния
Граф знаний	Узлы-сущности и рёбра-отношения
Узел	Вершина графа
Ребро	Связь между узлами
Триплет	Тройка «голова, отношение, хвост»
Онтология	Допустимые типы узлов и рёбер
Каноникализация	Приведение разных написаний к одному виду
Vault	Папка Obsidian с Markdown-заметками
Markdown	Простой текстовый формат разметки
Frontmatter	YAML-блок метаданных в начале заметки
Wikilink	Ссылка `[[Заметка]]` внутри vault
Stub	Лёгкая заметка-заглушка для внешней ссылки
Coverage	Доля gold-сущностей или связей, попавших в граф
Orphan	Узел без ссылок
Unresolved link	Ссылка, не находящая цель
Retrieval	Поиск кандидатов
Дедупликация	Удаление одинаковых статей
Ранжирование	Сортировка по полезности
Elbow cutoff	Отсечение низкоимпактного хвоста по «локтю»
Fallback	Запасной движок или вариант
Sub-query	Уточняющий подзапрос
Precision	Доля правильного среди найденного

Термин	Что это значит
Recall	Доля найденного среди правильного
F1	Гармоническое среднее precision и recall
Exact	Точное совпадение после нормализации
Relaxed	Совпадение по Jaccard ≥ 0.5
Micro	Агрегирование TP/FP/FN по всем примерам
Macro	Среднее метрик по примерам
MRR	Среднее обратных рангов
Hits@k	Доля запросов с правильным ответом в top-k
nDCG	Метрика ранжирования с градацией релевантности
MAP	Среднее Average Precision
Triple F1	F1 по тройкам графа
QA	Проверка качества
CI	Автоматический запуск тестов и линтера
pytest	Фреймворк тестов Python
ruff	Линтер Python
SciERC	Бенчмарк scientific knowledge-graph construction
SOTA	State of the art, лучший известный результат

10.2. FAQ

Это заменяет ChatGPT? Нет. LitGraphAgent — не чат, а конвейер с фиксированными шагами. LLM здесь исполнитель операций, а не свободный собеседник.

Почему не просто отправить тему в ChatGPT? Потому что нужен воспроизводимый граф, ссылки, библиография, состояние и проверки. Чат не даёт структуру, не сохраняет `_state.json` и не проверяет целостность vault.

Почему локальная модель? Локальная модель не отправляет тексты в облако, дешевле в эксплуатации и работает без интернета после загрузки. Минус — качество и скорость.

Почему qwen3.5:4b, а не qwen3.5:3b? Модели `qwen3.5:3b` нет в реестре Ollama; попытка скачать её даёт `pull model manifest: file does not exist`. Поэтому используется 4B, а для сравнения — `qwen2.5:3b`.

Почему exact F1 такой низкий? Потому что система возвращает канонические концепты, а gold — сырые спаны. `word sense disambiguation (WSD) systems` и `word sense disambiguation` — одно и то же по смыслу, но не по строке. Relaxed и content-only показывают более честную картину.

Почему не считается relation F1? Пайплайн не предсказывает типы отношений SciERC. Считать relation F1 было бы фикцией. Это ограничение честно зафиксировано в отчёте и README.

Почему 20 абстрактов? Это regression sample, а не лидерборд. 20 хватает, чтобы ловить регрессии, но не для статистической значимости. До КП3 планируется 100 абстрактов и 3 seed.

Почему prompt v2 медленнее? Он просит 6–8 концептов вместо трёх примеров, поэтому модель генерирует больше текста. Время выросло с ~31 до ~59 минут на 20 абстрактов. Это trade-off качества и скорости.

Можно ли использовать другую модель? Да. Любой OpenAI-совместимый endpoint: DeepSeek, OpenAI, Groq, OpenRouter, vLLM. Достаточно поменять `.env`.

Можно ли использовать без Obsidian? Vault — это папка Markdown. Obsidian нужен только для graph view и удобной навигации. Без него файлы всё равно читаются.

Что будет, если LLM вернёт плохой JSON? Санитайзер попробует восстановить, затем retry, затем fallback. Если совсем плохо, статья может остаться без части полей, а QA покажет warning.

Что будет, если сеть упала? Retrieval сделает retry/backoff, затем пропустит кандидата. База может быть меньше, но запуск не упадёт.

Как проверить, что база не сломана? `litgraph --check --id "your_base"`. QA покажет errors и warnings.

Где хранится состояние? В `vaults/<id>/_state.json`. Запись атомарная.

Можно ли обновлять базу частями? Да, `--update` для этого и нужен. Старые определения не перезаписываются.

Почему в графе есть внешние заметки? Это stubs для цитирований, которых нет в базе. Они нужны, чтобы `[[ссылки]]` не повисали.

Как добавить новый движок? Описать в `search/tools.py`, добавить клиент, подключить в `search/engine.py`, добавить тест. Не встраивать условие в агента.

Как добавить нового агента? Роль, сервис, инстанс в `core.py`, узлы/рёбра в `workflow.py`, тест, docs, CODEOWNERS.

Почему в отчёте есть SOTA SciERC, если мы не сравниваемся? Это внешний ориентир, чтобы читатель понимал масштаб. Прямое сравнение некорректно: SOTA считает span NER/RE, а мы — канонические концепты.

Что делать, если модель не влезает в GPU? Использовать меньшую модель или квантизацию, закрыть другие процессы, уменьшить контекст и батч. На 6 ГБ GPU 4B-модель работает, но медленно.

Почему в проекте так много документов? Потому что разные читатели хотят разное: README для быстрого старта, ARCHITECTURE для инженеров, KP2_REPORT для защиты, KG_METRICS для метрик, этот гайд для полного погружения.

10.3. Приложение A. Пример `_state.json`

Этот пример показывает структуру, а не реальную базу. В реальном файле больше статей, концептов и полей.

```
{
  "base_id": "transformer_circuits",
  "topic": "Mechanistic Interpretability and Induction Heads",
  "requirements": "Peer-reviewed or high-impact preprints after 2021",
  "citation_format": "APA 7th",
  "overall_summary_en": "The literature converges on...",
  "papers": {
    "2401.00001": {
      "id": "2401.00001",
      "title": "Example Paper",
      "authors": ["A. Author", "B. Author"],
      "year": 2024,
      "venue": "arXiv",
      "doi_or_url": "https://arxiv.org/abs/2401.00001",
```

```
      "source_engine": "arxiv",
      "citation_count": 42,
      "influential_citation_count": 7,
      "citation_velocity": 10.5,
      "influential_ratio": 0.186,
      "objective_scientific_score": 13.2,
      "normalized_weight": 0.71,
      "summary_en": "The paper studies...",
      "key_findings_en": ["Finding 1", "Finding 2"],
      "methodology_en": "The authors...",
      "limitations_en": "Small sample...",
      "extracted_concepts": ["Induction Head", "Transformer"],
      "extracted_concepts_all": ["Induction Head", "Transformer", "Attention"],
      "formatted_citation": "Author, A. (2024). Example Paper..."
    }
  },
  "concepts": {
    "Induction Head": {
      "name": "Induction Head",
      "summary_en": "A circuit that...",
      "aggregated_score": 4.2,
      "normalized_weight": 0.83,
      "mentions": [
        {
          "concept_name": "Induction Head",
          "paper_title": "Example Paper",
          "page_number": 3,
          "context_snippet": "We identify an induction head..."
        }
      ]
    }
  },
  "edges": [
    {
      "source_title": "Example Paper",
      "target_title": "Another Paper",
      "relation_type": "cites"
    }
  ],
  "search_plan": {
    "primary": ["arxiv", "semantic_scholar"],
    "fallback": ["web"],
    "reason": "Fresh preprints plus citation metadata.",
    "source": "llm"
  },
  "created_at": "2026-11-09T10:00:00+00:00",
  "updated_at": "2026-11-09T10:05:00+00:00"
}
```


10.4. Приложение B. Пример заметки источника

Заметка источника — это обычный Markdown с frontmatter. В реальном vault название файла очищено от запрещённых символов и может быть обрезано с хэшем.

```
---
id: "2401.00001"
title: "Example Paper"
authors: ["A. Author"]
year: 2024
venue: "arXiv"
url: "https://arxiv.org/abs/2401.00001"
source_engine: "arxiv"
citation_count: 42
citation_velocity: 10.5
objective_score: 13.2
node_weight: 0.71
type: literature_note
```

```
---

# Example Paper

> 📊 **Scientometric Score:** `13.2` | **Continuous Weight:** `0.71`

## 📌 Formatted Citation

> Author, A. (2024). Example Paper...

## 🧠 Executive Summary (English)

The paper studies...

## 🔍 Key Findings & Contributions

- Finding 1
- Finding 2

## ⚙️ Methodology & Limitations

**Methodology:** The authors...

**Limitations:** Small sample...

## 🌐 Knowledge Graph Connections

**Concepts:** [[Induction Head]], [[Transformer]]

**Related Literature:** None

**References (Outward):** [[Another Paper]]

**Citations (Inward):** None

## 📄 Abstract

> ...
```

10.5. Приложение C. Пример концепт-хаба

Концепт-хаб собирает определение и все упоминания концепта. Если в базе был deep-режим, у упоминаний есть страницы и цитаты.

```
---
concept: "Induction Head"
aggregated_score: 4.2
node_weight: 0.83
type: concept_hub
---

# Concept: Induction Head

> 🌐 **Aggregated Score:** `4.2` | **Continuous Weight:** `0.83`

## 📖 Concept Definition & Consensus

A circuit that...

## 📍 Grounded Citations & Page References:

- [[Example Paper]] (**p. 3**) — *"We identify an induction head..."*
```

10.6. Приложение D. Пример QA-отчёта

QA-отчёт показывает summary, статистику и список issues. В реальном отчёте issues может быть больше; в примере только один warning.

```
{
  "summary": "0 errors, 2 warnings (notes=42, sources=12, concepts=28, links=96)",
  "stats": {
    "notes": 42,
    "sources": 12,
    "concepts": 28,
    "external_refs": 2,
    "links": 96,
    "unresolved_links": 0,
    "orphans": 2
  },
  "issues": [
    {
      "severity": "warning",
      "code": "orphan_note",
      "message": "Note has no inbound or outbound links",
      "path": "Concepts/Example.md"
    }
  ]
}
```

10.7. Приложение E. Промпт v2

Этот промпт используется в `paper_synthesis_prompt`. Он просит 6–8 концептов, требует явной поддержки в абстракте и ставит концепты первыми в JSON.

```
Synthesize the following paper strictly in English:
Title: ...
Authors: ...
Year: ...
Venue: ...
Abstract: ...

Concept extraction rules:
- Return 6-8 key scientific concepts that are explicitly mentioned in the abstract.
- Prefer specific methods, tasks, metrics, materials, datasets and technical terms.
- Prefer canonical multi-word names over bare acronyms.
- Avoid generic words such as system, approach, method, algorithm or result.
- Return each concept once.

Return strict JSON with concepts first:
{
  "extracted_concepts": ["Specific concept 1", "Specific concept 2", "Specific concept 3"],
  "summary_en": "Deep single-paragraph executive summary in English",
  "key_findings_en": ["Key finding 1", "Key finding 2", "Key finding 3"],
  "methodology_en": "Methodology summary in English",
  "limitations_en": "Limitations identified in English"
}
```

10.8. Приложение F. Шпаргалка команд

```
# окружение
litgraph --version
litgraph --team
litgraph --tools

# создать
litgraph --id "base" --topic "..." --requirements "..." --format "APA 7th" --max-papers 5

# обновить
litgraph --id "base" --update "..." --max-papers 3

# deep
litgraph --id "base" --update "..." --deep

# проверить
litgraph --check --id "base"

# список и удаление
litgraph --list
litgraph --delete --id "base"

# тесты
pytest -q
ruff check lit_graph tests benchmarks

# бенчмарк
python benchmarks/scierc_benchmark.py --data-dir ./raw_data --limit 20 --seed 42 --model
qwen3.5:4b --output benchmarks/results/run.json
python benchmarks/plot_results.py benchmarks/results/run.json
```

10.9. Что читать дальше

- `README.md` — быстрый старт и верхнеуровневые метрики.
- `docs/ARCHITECTURE.md` — архитектура и диаграммы.
- `docs/RETRIEVAL.md` — поиск и ранжирование.
- `docs/LLM_LAYER.md` — промпты и синтез.
- `docs/GRAPH_MODEL.md` — онтология и каноникализация.
- `docs/KG_METRICS.md` — метрики KG-бенчмарков и SOTA-ориентиры.
- `docs/kp2/KP2_REPORT.md` — отчёт КП2.
- `benchmarks/README.md` — протокол SciERC.

10.10. Заключение

Если вы дошли до конца, у вас есть полная карта проекта: зачем он нужен, из каких агентов состоит, как проходит статья от поиска до заметки, что именно измеряет бенчмарк и где лежит код. Этого достаточно, чтобы включиться в работу, объяснить систему на защите и не потеряться в первый день.

Главное, что стоит запомнить: LitGraphAgent — это конвейер, а не «магический ИИ»; LLM — вероятностный компонент, поэтому есть валидация, JSON-санитайзер, fallback-и и QA; граф знаний держится на каноникализации, ссылках и дедупликации; `_state.json` — источник правды между запусками; бенчмарк измеряет только то, что система реально предсказывает; 20 абстрактов — это regression sample, а не лидерборд.

Если понадобится продолжить проект, начните с `README.md`, затем откройте `docs/ARCHITECTURE.md` и `docs/kp2/KP2_REPORT.md`, а после — код агентов в том порядке, в котором они вызываются: Lead → Retrieval → Prompt → Graph → QA.

Контакты и ссылки: репозиторий <https://github.com/Icold21/LitGraphAgent>, команда AI-RIIdium, лицензия MIT, дата сборки гайда — ноябрь 2026. Спасибо, что дочитали. Дальше — только код, эксперименты и вопросы.